

Align Forward, Adapt Backward: Closing the Discretization Gap in Logic Gate Networks

Youngsung Kim¹

Abstract

In neural network models, soft mixtures of fixed candidate components (e.g., logic gates and sub-networks) are often used during training for stable optimization, while hard selection is typically used at inference. This raises questions about training-inference mismatch. We analyze this gap by separating forward-pass computation (hard selection vs. soft mixture) from stochasticity (with vs. without Gumbel noise). Using logic gate networks as a testbed, we observe distinct behaviors across four methods: Hard-ST achieves zero selection gap by construction; Gumbel-ST achieves near-zero gap when training succeeds but suffers accuracy collapse at low temperatures; Soft-Mix achieves small gap only at low temperature via weight concentration; and Soft-Gumbel exhibits large gaps despite Gumbel noise, confirming that noise alone does not reduce the gap. We propose CAGE (Confidence-Adaptive Gradient Estimation) to maintain gradient flow while preserving forward alignment. On logic gate networks, Hard-ST and Gumbel-ST with CAGE both achieve over 98% accuracy on MNIST and $\sim 58\%$ on CIFAR-10; Hard-ST with CAGE further maintains zero selection gap across all temperatures by construction, while Gumbel-ST without CAGE suffers a ~ 47 -point accuracy collapse.

1. Introduction

Many neural network architectures require selecting among discrete options during computation (Mohamed et al., 2020; Heuillet et al., 2024; Cai et al., 2025). Differen-

¹Department of Artificial Intelligence and Department of Electrical and Computer Engineering, Inha University, Incheon, Korea. Correspondence to: Youngsung Kim <y.kim@inha.ac.kr; yskim.ee@gmail.com>.

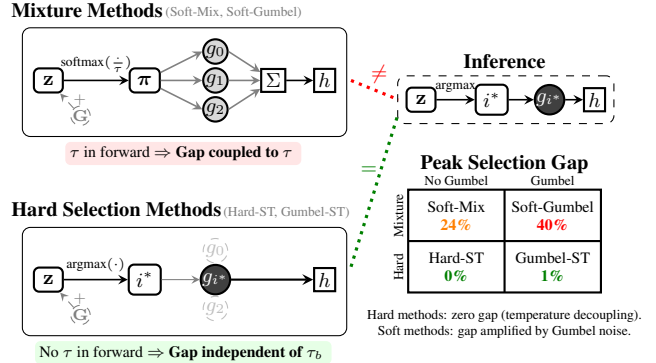


Figure 1. **Training-inference gap depends primarily on forward-pass structure.** **Left:** Mixture methods (top) compute weighted sums, enabling hedging; hard selection methods (bottom) commit to a single gate, matching inference. **Right-Top:** Inference uses argmax selection. **Right-Bottom:** Peak selection gap (2×2 factorial, MNIST-Binary, $L=6$, $\tau=2.0$). Hard methods achieve near-zero gap; soft methods show 24–40% gap, with Gumbel noise amplifying the gap.

tiable logic gate networks (Petersen et al., 2022; Kim, 2023; Yousefi et al., 2025) learn Boolean circuits by choosing among gate operations (AND, OR, XOR, etc.) at each neuron. Similar discrete selection arises in neural architecture search (Liu et al., 2019), mixture-of-experts (MoE) routing (Jacobs et al., 1991; Fedus et al., 2022), and vector quantization (Van Den Oord et al., 2017). When hard selection is required at inference, a fundamental challenge emerges: methods using soft mixtures during training but hard selection at deployment suffer from *training-inference mismatch*, while methods using hard selection throughout avoid this gap (Figure 1).

Recent work has achieved substantial gap reduction using the Straight-Through Gumbel-Softmax (Gumbel-ST) estimator (Jang et al., 2017; Maddison et al., 2017; Xie et al., 2019), which combines the Gumbel-Max trick (Gumbel, 1954) with straight-through gradient estimation (Bengio et al., 2013): hard selection (argmax with Gumbel noise) in the forward pass, with gradients computed through the continuous Gumbel-Softmax relaxation in the backward pass (Kim, 2023; Yousefi et al., 2025). This success has been attributed to *implicit Hessian regularization*: the hy-

pothesis that Gumbel noise smooths the loss landscape, leading to solutions that generalize better to discrete inference (Yousefi et al., 2025). We revisit this explanation and find it incomplete. Through controlled experiments, we show that methods with identical Gumbel noise exhibit markedly different gaps depending on whether their forward pass uses soft mixtures or hard selection. This suggests a simpler mechanism: gap reduction occurs because Gumbel-ST performs *hard selection during training*, aligning training computation with deployment.

We call this principle *forward alignment*: the forward pass during training should match the forward pass at deployment, eliminating training-inference mismatch. While conceptually straightforward, hard selection has been avoided due to concerns about biased gradient estimates from the straight-through approximation (Bengio et al., 2013) and the high variance of score-function alternatives like REINFORCE (Williams, 1992). Our central contribution is demonstrating that these gradient estimation challenges can be addressed *entirely in the backward pass* (through temperature scheduling and adaptive gradient estimation) without compromising forward alignment. This separation is possible because backward-pass temperature does not affect the forward output for hard selection methods, a property we call *temperature decoupling*.

Why does forward alignment matter? Our theoretical analysis reveals a fundamental asymmetry between mixture and hard selection methods. Mixture methods can minimize training loss by *hedging*: distributing weight across multiple options to produce a blended output, without any single option achieving low loss individually. At deployment, argmax forces commitment to one option, exposing this lack of preparation. Hard selection methods cannot hedge; they must *commit* to options that individually perform well, because only one option contributes to each forward pass. This distinction between hedging and commitment, formalized in Section 4, explains why forward alignment eliminates the *discretization gap* (the accuracy lost when a network’s continuous, differentiable training-time computation is discretized at deployment to a single argmax -selected gate evaluated as a Boolean function).

Contributions.

1. **Theoretical framework.** We decompose the training-inference gap into *selection gap* (method-dependent, reducible) and *computation gap* (gate- and input-dependent, irreducible), and prove that hard selection methods achieve zero or near-zero selection gap regardless of backward temperature (Corollary 2, Proposition 3). We formalize how mixture methods *hedge* and prove that hard selection eliminates it (Proposition 9).
2. **Empirical separation of causal factors.** Using a 2×2

factorial design crossing forward type (mixture vs. hard) with noise type (deterministic vs. Gumbel), we provide direct evidence that gap depends on forward structure, not gradient estimation. Critically, Soft-Gumbel and Gumbel-ST use identical noise distributions but exhibit dramatically different gaps, challenging the Hessian regularization hypothesis.

3. **Discovery of a failure mode.** We identify that Gumbel-ST suffers catastrophic accuracy collapse at low temperatures (dropping from $\sim 98\%$ to $\sim 50\%$), while deterministic Hard-ST remains stable. This has practical implications for temperature scheduling.
4. **CAGE algorithm.** We propose Confidence-Adaptive Gradient Estimation, which exploits temperature decoupling to adapt backward temperature based on selection confidence, enabling exploration early and exploitation late without affecting alignment guarantees.

2. Background and Related Work

2.1. Discrete Selection and the Training-Inference Gap

Notation. We write $\mathbf{z} \in \mathbb{R}^K$ for the selection logits over K candidates, $\pi_i(\tau) = \text{softmax}(\mathbf{z}/\tau)_i$ for the softmax weights, g_i for candidate i ’s output (a gate or expert), and $i^* = \arg \max_i z_i$ for the inference-time selection. Temperatures τ and τ_b denote the forward and backward temperature. The three test accuracies A^M , A^{soft} , and A^{hard} and the resulting selection and computation gaps are defined in Section 3.1. Soft gate evaluation uses each gate’s continuous (differentiable) formula, hard evaluation its Boolean $\{0, 1\}$ form; Table 15 lists both for all 16 gates. Finally, c_{ema} is the exponential moving average of the mean selection confidence.

Many neural network components must select among K discrete choices, each producing output g_i . Examples include Boolean gates in logic gate networks (Petersen et al., 2022), candidate operations in neural architecture search (Liu et al., 2019), and expert networks in mixture-of-experts (Zhou et al., 2022; Puigcerver et al., 2024; Jacobs et al., 1991). The standard approach parameterizes selection with logits $\mathbf{z} \in \mathbb{R}^K$, converted to probabilities via softmax (Maddison et al., 2017; Jang et al., 2017; Liu et al., 2019):

$$\pi_i(\tau) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}, \quad (1)$$

where temperature τ controls sharpness ($\tau \rightarrow 0$: one-hot; $\tau \rightarrow \infty$: uniform).

The challenge is that training and deployment handle these probabilities differently. During training, *soft selection* computes a differentiable weighted mixture $h_{\text{soft}} = \sum_i \pi_i(\tau) g_i$, enabling gradient-based optimization. At deployment, *hard selection* picks a single choice $h_{\text{hard}} = g_{i^*}$

where $i^* = \arg \max_i z_i$, as required for efficiency and interpretability (Liu et al., 2019). This structural difference between blending all choices and picking one creates the training-inference gap, even when individual g_i are computed identically in both phases.

A middle ground is *categorical sampling* via Gumbel-max (Jang et al., 2017; Maddison et al., 2017):

$$h_{\text{sample}} = g_k, \quad k = \arg \max_i (z_i + G_i), \quad (2)$$

where G_i are i.i.d. standard Gumbel variables (CDF: $F(x) = e^{-e^{-x}}$). The Gumbel-max theorem ensures $\mathbb{P}(k = i) = \text{softmax}(\mathbf{z})_i$. Crucially, this probability depends only on logits, not temperature, since argmax is scale-invariant (Gumbel, 1954). This property is central to our analysis.

2.2. Straight-Through Estimators

Hard selection is non-differentiable, but the straight-through (ST) estimator (Bengio et al., 2013) enables gradient-based training by decoupling forward and backward computation: the forward pass uses discrete selection, while the backward pass computes gradients through a continuous surrogate.

Gumbel-ST (Jang et al., 2017; Maddison et al., 2017) applies this principle to categorical sampling. The forward pass samples a discrete choice via (2), while the backward pass differentiates through $\tilde{\pi}(\tau) = \text{softmax}((\mathbf{z} + \mathbf{G})/\tau)$. Here, temperature τ controls gradient smoothness without affecting forward selection. Section 3 formalizes this as *temperature decoupling*.

2.3. Challenges of Hard Selection

If hard selection eliminates mismatch, why is it often avoided? The literature documents several concerns (Bengio et al., 2013; Jang et al., 2017; Liu et al., 2019). Sparse gradients mean only selected choices directly influence learning. Deterministic argmax may lock into suboptimal early choices. Discrete selections can flip abruptly, causing instability. Convergence is typically slower without smooth gradient landscapes (Zela et al., 2020). These concerns have led practitioners toward soft methods despite their mismatch problem, or toward domain-specific workarounds. Our approach resolves this tension by showing that backward-pass adaptation can address these optimization challenges while forward alignment preserves zero mismatch.

2.4. Related Work

The training-inference gap manifests across domains, with each community developing its own solutions. In neural architecture search, DARTS (Liu et al., 2019) exemplifies

the soft relaxation paradigm and the resulting discretization gap. Subsequent methods such as SNAS (Xie et al., 2019) and GDAS (Dong & Yang, 2019) introduce stochastic or near-hard operation selection to mitigate this mismatch. In large-scale mixture-of-experts models, hard routing has been predominantly adopted (Shazeer et al., 2017; Fedus et al., 2022), effectively avoiding training-inference mismatch. Similarly, VQ-VAE (Van Den Oord et al., 2017) employs hard codebook selection together with auxiliary commitment losses to stabilize learning. By contrast, differentiable logic gate networks (Petersen et al., 2022) typically rely on soft gate mixtures, which can introduce a soft-to-hard discrepancy at inference time. Kim (2023) introduced the Gumbel-ST framework for logic gate networks, relieving this discrepancy, and Yousefi et al. (2025) attribute its improvement to Hessian regularization acting as a smooth optimization objective.

Our analysis offers a unifying perspective: across these domains, methods succeed when forward computation aligns with deployment, regardless of the specific noise injection or regularization employed. Temperature scheduling has been explored via sparsifying small temperatures (Zhang & Ding, 2023) and decoupled forward-backward temperatures (Shah et al., 2024); our framework explains *why* such approaches succeed specifically for hard forward methods. A complementary line of work improves the straight-through estimator itself: ReinMax (Liu et al., 2023) raises the gradient approximation from first to second order. Because CAGE adapts the backward *temperature* rather than the estimator’s order, the two are orthogonal and can be combined.

3. Forward Alignment Framework

Section 2 introduced soft and hard selection. We now characterize where the training-inference gap arises and which design choices eliminate it.

3.1. Gap Decomposition

The gap between training and deployment has two potential sources: (1) the selection mechanism (mixture vs. argmax), and (2) the gate evaluation (soft vs. hard; see Appendix K.1 for definitions). To isolate these, we define three accuracy measures on the test set.

Let $i^* = \arg \max_i z_i$ denote the selected gate. A^M uses method M ’s forward computation, matching the training setting. A^{soft} uses argmax selection with soft gate evaluation $g_{i^*}^{\text{soft}}$. A^{hard} uses argmax selection with hard gate evaluation $g_{i^*}^{\text{hard}}$, matching the deployment setting.

We refer to the total gap $A^M - A^{\text{hard}}$ (the method’s training-time forward versus the discretized deployment forward) as

the *discretization gap*, which decomposes as:

$$\underbrace{A^M - A^{\text{hard}}}_{\text{Total Gap}} = \underbrace{A^M - A^{\text{soft}}}_{\text{Selection Gap}} + \underbrace{A^{\text{soft}} - A^{\text{hard}}}_{\text{Computation Gap}} \quad (3)$$

Selection gap isolates the effect of selection mechanism. For mixture methods, training uses $\sum_i \pi_i g_i^{\text{soft}}$ while deployment uses single gate $g_{i^*}^{\text{soft}}$; these differ. For hard selection methods, training already uses $g_{i^*}^{\text{soft}}$, so $A^M = A^{\text{soft}}$ and selection gap is zero.

Computation gap isolates the effect of gate evaluation, comparing soft versus hard gates for the same selected gate. It depends on the selected gate’s soft-vs-hard discrepancy and the input distribution (zero for binary inputs, gate-dependent for continuous; Appendix K), and arises from gate evaluation rather than the selection mechanism. More broadly it is a relaxation gap, arising only for components with distinct soft (training) and hard (deployment) forms (e.g., logic gates, quantized experts) and vanishing otherwise, as for standard MoE experts, whose total gap reduces to selection gap (Appendix J).

The decomposition reveals an asymmetry: selection gap is method-dependent and reducible, whereas computation gap is gate- and input-dependent, irreducible, yet method-independent. Forward alignment targets the selection gap; the computation gap is an irreducible baseline we characterize but do not aim to remove. (In the remainder of the paper, we write g_i without superscript; the soft/hard distinction is relevant only for computation gap.)

3.2. Forward Alignment and Temperature Decoupling

Forward alignment. Figure 1 organizes methods along two axes: selection type (mixture vs. hard) and noise type (deterministic vs. Gumbel). The key insight is that selection type (row), not noise type (column), determines selection gap. If training uses argmax selection (hard selection methods), it matches deployment exactly, regardless of whether Gumbel noise is present. We call such methods *forward-aligned*.

This yields predictions that contradict prior work attributing gap reduction to Gumbel noise (Yousefi et al., 2025): (1) Hard-ST should have smaller gap than Soft-Gumbel, despite lacking Gumbel noise; (2) adding Gumbel noise to mixture methods (Soft-Mix $\rightarrow$ Soft-Gumbel) should not reduce selection gap. Section 6 tests these predictions.

Temperature decoupling. Forward alignment provides a practical benefit beyond gap reduction. In mixture methods, temperature τ controls the forward output through $\pi_i(\tau) = \text{softmax}(\mathbf{z}/\tau)_i$, so changing τ alters the mixture weights and thereby the selection gap. Temperature and gap are coupled.

In hard selection methods, the forward pass uses argmax, which is scale-invariant: $\arg \max_i z_i = \arg \max_i (z_i/\tau)$ for any $\tau > 0$. Temperature enters only the backward pass through the ST surrogate gradient. We call this the *backward temperature* τ_b . Since τ_b does not affect the forward output, it does not affect the selection gap. Temperature and gap are decoupled.

This decoupling enables adaptive scheduling: high τ_b early for exploration, low τ_b late for exploitation, without affecting alignment. Section 5 develops this into the CAGE algorithm.

4. Theoretical Analysis

Section 3 decomposed the training-inference gap into selection gap (method-dependent) and computation gap (gate- and input-dependent). We now provide rigorous bounds on these gaps, analyze why different methods produce different solutions, and characterize how temperature affects convergence. Proofs appear in Appendix C.

4.1. Node-Level Gap Bounds

Consider a selection node choosing among K gates with outputs $\{g_0, \dots, g_{K-1}\}$ based on logits $\mathbf{z}$. Let $i^* = \arg \max_i z_i$ denote the inference-time selection. We analyze the selection gap Δh_{sel} from Section 3.1, which captures mismatch due to different selection mechanisms.

Proposition 1 (Gumbel-Max Theorem (Gumbel, 1954)). *For $G_i \stackrel{\text{iid}}{\sim} \text{Gumbel}(0, 1)$:*

$$\mathbb{P}(\arg \max_i (z_i + G_i) = j) = \text{softmax}(\mathbf{z})_j. \quad (4)$$

This probability is temperature-independent since $\arg \max$ is scale-invariant. (Proof sketch in Appendix C.1)

Corollary 2 (Gumbel-ST: Selection Gap). *For Gumbel-ST with $k = \arg \max_i (z_i + G_i)$, the expected selection gap is*

$$\mathbb{E}[\Delta h_{\text{sel}}] = \sum_{i \neq i^*} p_i (g_i - g_{i^*}), \quad p_i = \text{softmax}(\mathbf{z})_i. \quad (5)$$

This gap is determined entirely by the logit-induced distribution, independent of any temperature used in gradient computation. (Proof in Appendix C.2)

Proposition 3 (Hard-ST: Zero Selection Gap). *For Hard-ST, $\Delta h_{\text{sel}} = 0$ exactly, since both training and inference output g_{i^*} where $i^* = \arg \max_i z_i$.*

Proposition 4 (Mixture Methods: Selection Gap). *For Soft-Mix and Soft-Gumbel, the selection gap is*

$$\Delta h_{\text{sel}} = \sum_{i \neq i^*} w_i(\tau) (g_i - g_{i^*}), \quad (6)$$

where $w_i = \pi_i(\tau)$ for Soft-Mix or $w_i = \tilde{\pi}_i(\tau)$ for Soft-Gumbel. This gap depends on temperature τ through the weights. (Proof in Appendix C.4)

Theorem 5 (Forward Alignment Principle). *Selection gap depends on whether training-time selection matches inference:*

- (i) *Hard-ST uses identical selection at training and inference, achieving $\Delta h_{sel} = 0$ exactly.*
- (ii) *Gumbel-ST samples a gate that matches i^* with probability p_{i^*} , so $\mathbb{E}[\Delta h_{sel}] \rightarrow 0$ as $p_{i^*} \rightarrow 1$.*
- (iii) *Mixture methods compute weighted averages rather than selecting, yielding $\Delta h_{sel} \neq 0$ whenever gates produce distinct outputs.*

Theorem 6 (Unified Selection Gap Bound). *For any method with effective weights $w_i \geq 0$, $\sum_i w_i = 1$:*

$$|\Delta h_{sel}| \leq (1 - w_{i^*}) \cdot \Delta_{\max}, \quad (7)$$

where $\Delta_{\max} = \max_i |g_i - g_{i^*}|$. (Proof in Appendix C.5)

For Hard-ST, $w_{i^*} = 1$ yields zero gap. For Gumbel-ST, $w_{i^*} = p_{i^*}$ yields a bound of $(1 - p_{i^*})\Delta_{\max}$ that tightens as confidence increases. For mixture methods, the bound depends on temperature through $w_{i^*}(\tau)$.

4.2. Network-Level Analysis

Networks contain thousands of selection nodes. For the GroupSum architecture (Petersen et al., 2022; Yousefi et al., 2025) where class logit $y_c = \sum_{n \in \mathcal{G}_c} h_n$, the logit gap decomposes linearly: $\Delta y_c = \sum_{n \in \mathcal{G}_c} \Delta h_n$.

Proposition 7 (Loss Gap Bound). *Let $\mathbf{y}^M$ denote the network output logits using method M 's forward pass, and $\mathbf{y}^{hard}$ the output using hard selection with hard gate evaluation. For loss $\mathcal{L}(\mathbf{y})$ with logit gap $\Delta = \mathbf{y}^M - \mathbf{y}^{hard}$:*

$$|\Delta \mathcal{L}| \leq \|\nabla_{\mathbf{y}} \mathcal{L}\|_p \cdot \|\Delta\|_q + O(\|\Delta\|^2). \quad (8)$$

(Proof in Appendix C.6)

This bound shows that loss gap is controlled by prediction uncertainty (gradient norm) and total selection gap (logit gap). When nodes fully commit, the logit gap vanishes and so does the loss gap. However, the Hölder-based bound (Rudin, 1987) is worst-case and may be loose in practice, as individual node gaps can partially cancel. We therefore evaluate actual network-level gaps empirically in Section 6.

For Hard-ST, selection gap is exactly zero at every node, so the network-level selection gap is exactly zero by construction.

4.3. Gradient Analysis

The gap analysis establishes *what* differs at inference. We now analyze gradients to understand *how* training dynamics differ.

Proposition 8 (Gradient Structure). *All four methods compute logit gradients as:*

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\delta}{\tau_b} \cdot w_j \cdot (g_j - \bar{h}), \quad (9)$$

where $w_j = \text{softmax}(\mathbf{z}/\tau_b)_j$, $\bar{h} = \sum_i w_i g_i$ is the surrogate mixture, and $\delta = \partial \mathcal{L} / \partial h$ is the upstream gradient evaluated at the method's forward output. For hard selection methods, argmax is non-differentiable; the straight-through estimator uses the soft surrogate for gradient computation while preserving δ from the hard forward output. (Proof in Appendix C.7)

Although all methods share the same gradient form, they differ in where $\delta = \partial \mathcal{L} / \partial h$ is evaluated. For mixture methods, the loss is computed on $h = \bar{h}$, coupling δ to the soft mixture. For hard selection methods, the loss is computed on $h = g_k$ (the selected gate), so δ reflects the discrete output used at inference.

This distinction determines low-temperature behavior. For mixture methods, forward and backward share the same temperature τ . As $\tau \rightarrow 0$, the mixture collapses ($\bar{h} \rightarrow g_{i^*}$), reducing selection gap structurally. However, gradients also vanish: the winner's $(g_{i^*} - \bar{h}) \rightarrow 0$; non-winners' $w_j \rightarrow 0$. This creates a dilemma: low τ reduces gap but cripples training.

Hard selection methods avoid this dilemma through temperature decoupling. The forward pass is τ -independent, so selection gap depends on learned confidence, not temperature. The backward temperature τ_b controls only gradient distribution. However, τ_b must still be set appropriately: if too low, gradients collapse, training fails, and selection gap remains large (as observed with Gumbel-ST at low temperature). This motivates CAGE (Section 5), which adapts τ_b based on training progress.

Proposition 9 (Competitive Gate Selection). *Unlike standard neurons where parameters update independently, selection nodes induce competition via the gradient term $(g_j - \bar{h})$ from Proposition 8. Gates outperforming the current mixture receive increased weight; underperforming gates are suppressed. This competition drives weight concentration over training. (Analysis in Appendix C.8)*

The competition leads to different outcomes. Mixture methods have achievable output space $[0, 1]$, so hedging (diffuse weights producing $\bar{h} \notin \{0, 1\}$) can reduce loss even if no single gate performs well. Hard methods have achievable output space $\{0, 1\}$, so only the selected gate affects loss, forcing commitment. A node that hedges produces selection gap at inference when $\bar{h} \neq g_{i^*}$. Hard methods cannot hedge, eliminating this gap.

We verify this empirically in Section 6, measuring selection confidence $\max_i \pi_i$ across training.

4.4. Convergence and Temperature

Proposition 10 (Convergence Scaling). *For a single selection node with learning rate η and backward temperature τ_b , assuming approximately constant upstream gradients, steps to reach confidence $1 - \epsilon$ scale as $T(\tau_b) = O(\tau_b \cdot \log(1/\epsilon)/\eta)$. (Proof in Appendix C.9)*

Convergence time is linear in τ_b . High τ_b produces diffuse weights w_j , distributing gradient across all gates and slowing winner emergence. Low τ_b concentrates gradient on the current winner, accelerating convergence but risking commitment to suboptimal gates before sufficient exploration.

This suggests adaptive scheduling: high τ_b early for broad exploration, decreasing over training for faster convergence. For hard selection methods, such adaptation affects only optimization dynamics; Proposition 3 guarantees selection gap remains zero regardless of τ_b schedule.

5. CAGE: Confidence-Adaptive Gradient Estimation

Corollary 2 and Proposition 3 establish that hard selection methods decouple backward temperature τ_b from selection gap. Proposition 10 shows convergence time scales linearly with τ_b . This creates an opportunity: adapt τ_b throughout training to accelerate convergence as confidence increases, without affecting forward alignment.

5.1. Exploration Mechanisms

Hard selection methods offer two distinct exploration mechanisms.

Forward exploration. Gumbel-ST selects gate i with probability $p_i = \text{softmax}(\mathbf{z})_i$, occasionally selecting non-argmax gates. This is controlled by logit magnitudes, not temperature. Hard-ST has no forward exploration (deterministic argmax).

Backward exploration. The gradient to logit z_j includes factor $\pi_j(\tau_b)$ from the ST surrogate. High τ_b distributes gradients across all gates; low τ_b concentrates gradients on the current winner. This is controlled by τ_b and can be adapted without affecting selection gap.

5.2. The CAGE Algorithm

CAGE adapts τ_b based on network-wide selection confidence:

$$c(t) = \frac{1}{N} \sum_{n=1}^N \max_i \text{softmax}(\mathbf{z}_n(t))_i, \quad (10)$$

where N is the number of selection nodes and t is the training step. Confidence has uniform floor $1/K$ and grows

toward 1 as nodes commit; in practice c starts somewhat above the floor, since a random maximum over K gates is already non-uniform.

CAGE maps confidence to temperature via linear interpolation:

$$\tau_b(t) = \tau_{\max} - (\tau_{\max} - \tau_{\min}) \cdot c_{\text{ema}}(t), \quad (11)$$

where c_{ema} is an exponential moving average (Kingma & Ba, 2015) with coefficient β , initialized to $1/K$. Low confidence yields high τ_b for broad gradient distribution; high confidence yields low τ_b for faster convergence. Confidence can optionally be normalized to $[0, 1]$ as $(c_{\text{ema}} - 1/K)/(1 - 1/K)$ so that τ_b spans the full $[\tau_{\min}, \tau_{\max}]$ range; our sensitivity study (Appendix I) uses this normalized form, with no material change in behavior. Full pseudocode is provided in Algorithm 1 (Appendix D).

CAGE applies to both Hard-ST and Gumbel-ST. With Hard-ST, selection gap is exactly zero and exploration occurs purely through adaptive gradient distribution. With Gumbel-ST, low backward temperature causes gradient concentration on randomly sampled gates, leading to training collapse (Section 6.3). CAGE prevents this by maintaining high τ_b while confidence is low, only reducing temperature as gates commit. Forward stochasticity provides additional exploration while the selection gap remains small. Computational overhead is negligible: $\max_i \text{softmax}(\mathbf{z}_n)_i$ reuses the softmax computed for gradient estimation.

6. Experiments

6.1. Setup

We validate forward alignment by measuring selection gap across temperatures, convergence speed, gate commitment (per-layer selection confidence $\max_i \pi_i$), and the effect of backward temperature adaptation via CAGE.

Methods. We compare six methods: four from our 2×2 factorial (Soft-Mix, Soft-Gumbel, Hard-ST, Gumbel-ST) plus Hard-ST+CAGE and Gumbel-ST+CAGE (Section 5).

Datasets. We use three datasets that isolate different gap components. **MNIST** (LeCun et al., 1998): Continuous pixel values in $[0, 1]$; both selection gap and computation gap are observable. **MNIST-Binary**: Pixels thresholded to $\{0, 1\}$; computation gap vanishes since soft gates equal hard gates on binary inputs, isolating *pure selection gap*. **CIFAR-10-Binary** (Krizhevsky, 2009): Each input channel is binarized using 31 evenly-spaced thresholds over $[0, 255]$; binary inputs isolate pure selection gap on a harder benchmark (3-channel natural images).

Architecture and Training. We use flat (equal-width) logic gate networks with $L \in \{4, 5, 6\}$ layers, $K = 16$ candidate gates per node, and 64k neurons per layer (128k

for CIFAR-10). Training uses Adam optimizer with batch size 512. Learning rate is 10^{-2} for MNIST (50k iterations) and 5×10^{-2} for CIFAR-10 (60k iterations). We sweep temperatures $\tau \in \{0.05, 0.1, 0.5, 1.0, 2.0\}$ with 3 random seeds per configuration. CAGE uses $\tau_{\max} = 3.0$, $\tau_{\min} = 0.5$, $\beta = 0.99$. We evaluate training accuracy, test accuracy, and selection gap every 500 iterations. For stable training, we derive a variance-based temperature for the GroupSum layer that produces well-conditioned softmax outputs at initialization (Appendix G). Full hyperparameters are provided in Appendix H.1.

6.2. Forward Alignment Eliminates Selection Gap

Our central claim is that forward-pass structure determines the selection gap. Table 1 tests this on MNIST-Binary ($L=6$), where binary inputs eliminate computation gap, isolating pure selection gap. We report final gap (at convergence) and peak gap (maximum during training).

The results reveal clear patterns supporting forward alignment:

Hard-ST achieves exactly zero selection gap at all temperatures, validating Proposition 3. This is the central result: because Hard-ST uses argmax in the forward pass during both training and inference, there is no selection mismatch by construction. The zero gap holds from $\tau = 0.05$ to $\tau = 2.0$, confirming this is a fundamental property of forward alignment, not an artifact of temperature choice.

Temperature decoupling holds for hard methods. Both Hard-ST (0% everywhere) and Gumbel-ST (0–1.1% at $\tau \geq 0.5$) achieve near-zero gap when training succeeds. The presence of Gumbel noise in the backward pass does not affect the gap for hard forward methods, confirming that backward-pass noise is decoupled from forward-pass alignment. This is a key prediction of our theoretical framework that distinguishes forward alignment from Hessian smoothing.

Soft methods show temperature-dependent gaps. Soft-Mix exhibits near-zero gap at low temperatures but develops significant gap at $\tau = 2.0$ (0.6% final, 24% peak). Soft-Gumbel shows gaps at all temperatures (0.2–2.1% final, 2–40% peak), with Gumbel noise amplifying the hedging behavior. The consistent pattern (Soft-Gumbel > Soft-Mix at every temperature) confirms that Gumbel noise encourages hedging by preventing commitment to individual gates.

Gumbel-ST fails catastrophically at low temperature. At $\tau \leq 0.1$, Gumbel-ST shows large negative gaps (−40% to −59%), indicating A^M far below A^{soft} . These negative values are a measurement artifact of training failure; we analyze this regime in Section 6.3.

Figure 2(a) visualizes the selection gap dynamics over train-

Table 1. Selection gap (%) on MNIST-Binary ($L=6$). Format: Final / Peak (omitted for training failures marked with †). Hard-ST achieves exactly zero gap at all temperatures. Soft methods show temperature-dependent gaps. Negative values indicate training failure (A^M below A^{soft}). **Bold**: zero selection gap.

Method	$\tau=0.05$	$\tau=0.1$	$\tau=0.5$	$\tau=1.0$	$\tau=2.0$
Soft-Mix	0.0 / 0	0.0 / 0	0.0 / 0.1	0.0 / 0.1	0.6 / 24
Soft-Gumbel	0.7 / 2	0.2 / 2	0.2 / 7	0.2 / 17	2.1 / 40
Hard-ST	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0
Gumbel-ST	−40 [†]	−59 [†]	0.0 / 0.1	0.0 / 0.3	0.0 / 1.1
Hard-ST+CAGE	0 / 0	0 / 0	0 / 0	0 / 0	0 / 0
Gumbel-ST+CAGE	0.0 / 4.5	0.1 / 9	0.0 / 3.5	0.0 / 1.6	0.0 / 2.2

[†] Training failure: optimization collapses, so the stochastic forward A^M falls below deterministic A^{soft} on the same logits.

Table 2. Temperature robustness: worst accuracy / range (%) across $\tau \in \{0.05, 0.1, 0.5, 1.0, 2.0\}$, $L=6$. Gumbel-ST suffers catastrophic failure at low τ across all datasets; CAGE eliminates this sensitivity entirely. **Bold**: worst performance (failure case); underline: best range (most robust).

Method	MNIST	M-Binary	C-10-Binary
Soft-Mix	97.1 / 1.4	97.2 / 1.1	50.7 / 7.5
Soft-Gumbel	85.9 / 12.0	85.6 / 12.2	52.6 / 5.4
Hard-ST	97.1 / 1.3	97.1 / 1.1	50.9 / 7.2
Gumbel-ST	52.1 / 46.3	51.0 / 47.3	18.9 / 39.5
Hard-ST+CAGE	98.4 / <u>0.1</u>	98.1 / <u>0.0</u>	57.6 / <u>0.5</u>
Gumbel-ST+CAGE	98.4 / <u>0.1</u>	98.2 / <u>0.1</u>	58.0 / <u>0.6</u>

ing iterations. Hard-ST maintains exactly zero gap throughout training (blue line perfectly at 0%), demonstrating that forward alignment provides immediate and sustained elimination of selection mismatch. Soft-Gumbel peaks at $\sim 17\%$ around iteration 5,000 before gradually recovering as the network commits to gates, but this recovery is incomplete, leaving residual gap at convergence. Interestingly, at low temperatures, Gumbel-ST shows transient *negative* gaps early in training, reflecting periods where stochastic sampling disrupts learning (see Section 6.3). These patterns hold across all temperatures and layers; see Figure 4 in the appendix for gap heatmaps over the full (layer $\times$ τ) grid.

6.3. Gumbel-ST Failure at Low Temperature

Figure 2(b) reveals a critical failure mode distinct from the selection gap: Gumbel-ST accuracy collapses from $\sim 98\%$ ($\tau \geq 0.5$) to $\sim 50\%$ ($\tau = 0.05$), reducing performance to near-random guessing on MNIST. Soft-Gumbel also suffers, dropping to $\sim 85\%$ at $\tau = 0.05$, though less severely. In contrast, Hard-ST and all CAGE methods remain stable at 97–98.5% across the entire temperature range.

This failure occurs because low τ concentrates the surrogate gradient on the sampled gate. When Gumbel noise selects a suboptimal gate early in training, the concen-

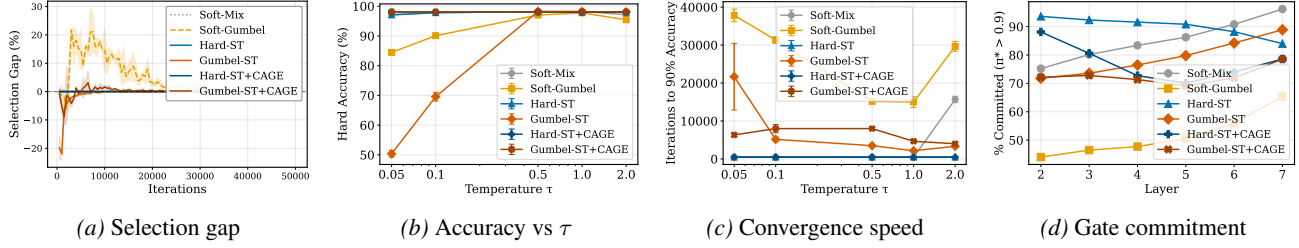


Figure 2. Main results (MNIST-Binary, $L=6$, $\tau=1.0$ unless varied). (a) Selection gap during training: Hard-ST achieves exactly zero; soft methods show non-zero gaps. (b) Test accuracy vs temperature: Hard-ST and CAGE remain stable; Gumbel-ST fails at low τ . (c) Iterations to convergence: Hard-ST converges significantly faster than soft methods. (d) Gate commitment by layer (mean selection confidence $\max_i \pi_i$): Hard-ST commits; Soft-Gumbel hedges, especially in early layers.

trated gradient reinforces that poor choice, creating a self-reinforcing loop that prevents recovery. Hard-ST avoids this via deterministic argmax selection: the same gate receives gradient updates in every forward pass, so learning signal consistently improves the best available option rather than reinforcing random early choices.

The negative selection gaps at $\tau \leq 0.1$ (Table 1) are a measurement artifact of this collapse, not a genuine advantage of A^{soft} over A^M . Recall that A^M for Gumbel-ST re-samples Gumbel noise at test time ($\arg \max_i (z_i + G_i)$), whereas A^{soft} applies deterministic $\arg \max_i z_i$ to the same learned logits. When the logits carry little signal (low confidence), test-time noise easily flips the top selection, so stochastic re-evaluation is noisier than deterministic selection on the same uninformative logits, giving $A^M < A^{\text{soft}}$. We label this a training failure because optimization has collapsed under low τ ; in this regime the selection gap, which quantifies selection-mechanism mismatch, no longer measures optimization quality. We emphasize that the failure stems from low τ *without an escape mechanism*, not from low confidence early in training per se: an adaptive schedule that begins even at $\tau_b=0.05$ but is allowed to rise as learning progresses (Reverse-CAGE) trains stably to the same final accuracy (Appendix I).

Table 2 confirms this pattern across all three datasets. Gumbel-ST exhibits catastrophic accuracy variation (39–47% range across temperatures), while CAGE variants remain stable (0.0–0.6% range). On CIFAR-10-Binary, Gumbel-ST drops to just 18.9%. CAGE prevents this failure by maintaining high temperature until gates commit.

6.4. Convergence Speed

Figure 2(c) reveals a marked difference in convergence speed: Hard-ST converges in approximately 500 iterations and maintains this speed across all temperatures, appearing as a flat line near the bottom of the plot. In contrast, Soft-Gumbel requires 15,000–38,000 iterations depending on temperature, making Hard-ST 30–76 $\times$ faster. This directly contradicts claims that Gumbel noise accelerates training

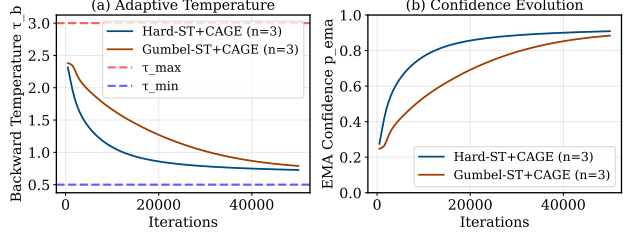


Figure 3. CAGE adaptation dynamics (MNIST-Binary, $L=6$). (a) Backward temperature over training: τ_b decreases from $\tau_{\max}$ as confidence rises. (b) EMA confidence over training: increases from low confidence (~ 0.25) toward commitment (~ 0.9). Results averaged over $n=3$ random seeds.

via Hessian regularization (Yousefi et al., 2025).

The speed advantage stems from Hard-ST’s gradient structure: deterministic argmax selection concentrates all learning signal on improving the current best gate, enabling rapid commitment. Gumbel methods dilute gradients across stochastically sampled gates, introducing variance that slows convergence. Notably, Hard-ST’s convergence time is temperature-invariant (~ 500 iterations at any τ), reflecting temperature decoupling. Soft-Gumbel shows a U-shaped pattern: slowest at extreme temperatures ($\sim 38,000$ at $\tau = 0.05$, $\sim 30,000$ at $\tau = 2.0$) and fastest at moderate temperatures.

6.5. CAGE Eliminates Temperature Sensitivity

Figure 3 shows the dynamics of CAGE adaptation over 50,000 training iterations. In panel (a), the backward temperature τ_b starts at ~ 2.4 (near $\tau_{\max} = 3.0$) to encourage exploration, then decreases as selection confidence increases. Hard-ST+CAGE drops rapidly to ~ 1.0 within 5,000 iterations and continues decreasing to ~ 0.7 by convergence; Gumbel-ST+CAGE decreases more gradually, reaching ~ 0.8 . Notably, neither method reaches $\tau_{\min} = 0.5$, indicating that the networks maintain some gradient smoothing throughout training rather than converging to hard gradients.

Panel (b) shows the corresponding confidence evolution.

Both methods start at low confidence (~ 0.25) and increase as gates commit. Hard-ST+CAGE reaches ~ 0.9 confidence by iteration 50,000, while Gumbel-ST+CAGE reaches ~ 0.85 . The faster confidence growth in Hard-ST+CAGE reflects deterministic selection’s tendency toward rapid commitment.

CAGE proves highly effective at eliminating temperature sensitivity. On MNIST-Binary, CAGE reduces Gumbel-ST’s accuracy range from 47.3 percentage points to just 0.1 percentage points (Table 2). On CIFAR-10-Binary, where Gumbel-ST collapses to 18.9%, Gumbel-ST+CAGE maintains 58.0% accuracy with only 0.6 percentage points variation, slightly outperforming Hard-ST+CAGE (57.6%), suggesting that Gumbel noise provides beneficial exploration when stabilized by CAGE.

CAGE provides different benefits depending on the base method. For Hard-ST, improvement is modest (already robust due to temperature decoupling). For Gumbel-ST, CAGE is critical: it reduces accuracy variation from 39–47 percentage points to under 1 percentage point across all datasets (Table 2).

6.6. Gate Commitment Explains Hedging

Proposition 9 predicts that mixture methods can hedge while hard selection methods must commit. Figure 2(d) validates this empirically: Hard-ST maintains high commitment (84–93%) across all layers, while Soft-Gumbel shows markedly lower commitment (45–66%). This difference directly explains the selection gap: Soft-Gumbel’s hedging succeeds during training but fails at inference when argmax selects a single gate.

The layer-wise patterns reveal different commitment mechanisms. Hard-ST commits strongly from early layers, reflecting structural commitment forced by the hard forward pass. In contrast, Soft-Mix and Gumbel-ST show increasing commitment toward the output (72–96%), driven by stronger gradient signal near the loss.

Interestingly, Soft-Mix achieves high commitment (75–96%) despite soft forward computation. Without Gumbel noise, gradient flow concentrates on the dominant gate, and the network naturally commits. This suggests Gumbel noise, not soft forward computation alone, is the primary enabler of hedging.

Gumbel-ST shows intermediate commitment (72–90%). Although each forward pass selects a single gate, stochastic sampling provides exploration, allowing more diversity than deterministic Hard-ST.

Hard-ST shows elevated XOR/XNOR usage ($\sim 11\%$ vs 6–8% for other methods), suggesting deterministic selection encourages commitment to more expressive gates.

Soft and Gumbel-based methods favor unitary gates (A, B, $\neg A$, $\neg B$), which pass specific inputs like skip connections, potentially explaining higher commitment in later layers. With CAGE, commitment decreases slightly, indicating that adaptive temperature encourages more exploration. See Appendix H.6 for details. All findings are statistically significant ($p < 0.05$); see Appendix H.5.

7. Conclusion

We analyzed trainable discrete selection embedded in neural networks and confirmed forward alignment as the key principle for eliminating training-inference mismatch. This principle is simple but not trivial, as prior work has not clearly investigated the causes of gap reduction. Our results showed that Hard-ST achieved exactly zero selection gap at all temperatures, while soft methods exhibited large peak gaps. Critically, Soft-Gumbel and Gumbel-ST used identical Gumbel noise but exhibited markedly different gaps: forward computation structure, not noise distribution, determined the gap. For practitioners, Hard-ST can be the default without careful tuning; Gumbel-ST with CAGE is recommended when exploration and stability are critical. While our experiments focused on logic gate networks, the principle may extend to other architectures requiring discrete selection at deployment (e.g., neural architecture search, mixture-of-experts, and vector quantization). A mixture-of-experts experiment confirms the decomposition across architectures (selection gap zero for hard routing, nonzero for soft; Appendix J); extending CAGE to neural architecture search is a natural next step.

Acknowledgements

We thank anonymous reviewers for their constructive feedback. This research was supported by the National Research Foundation (NRF) funded by the Korean government (MSIT) (RS-2024-00455784). This work was supported in part by Inha University Research Grant.

Impact Statement

This paper analyzes the training-inference gap in discrete selection networks and proposes methods to reduce it. Our contributions are primarily theoretical and methodological, focusing on understanding gradient estimation for discrete neural network components.

The techniques studied—logic gate networks, straight-through estimators, and temperature scheduling—are general-purpose tools applicable across many domains. We do not foresee specific negative societal consequences arising directly from this work. The potential applications (efficient neural networks, neural architecture search, mixture-

of-experts routing) carry the same broad implications as machine learning research in general.

References

- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Cai, W., Jiang, J., Wang, F., Tang, J., Kim, S., and Huang, J. A survey on mixture of experts in large language models. *IEEE Transactions on Knowledge and Data Engineering*, 37(7):3896–3915, 2025.
- Chen, X., Xie, L., Wu, J., and Tian, Q. Progressive differentiable architecture search: Bridging the depth gap between search and evaluation. In *IEEE International Conference on Computer Vision (ICCV)*, 2019.
- Chu, X., Zhou, T., Zhang, B., and Li, J. Fair DARTS: Eliminating unfair advantages in differentiable architecture search. In *European Conference on Computer Vision (ECCV)*, 2020.
- Cohen, J. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.
- Dong, X. and Yang, Y. Searching for a robust neural architecture in four GPU hours. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Gumbel, E. J. *Statistical Theory of Extreme Values and Some Practical Applications: A Series of Lectures*. National Bureau of Standards Applied Mathematics Series 33. U.S. Government Printing Office, 1954.
- Heuillet, A., Nasser, A., Arioui, H., and Tabia, H. Efficient automation of neural network design: A survey on differentiable neural architecture search. *ACM Computing Surveys*, 56(11):1–36, 2024.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with Gumbel-Softmax. In *International Conference on Learning Representations (ICLR)*, 2017.
- Jordan, M. I. and Jacobs, R. A. Hierarchical mixtures of experts and the em algorithm. *Neural Computation*, 6(2):181–214, 1994.
- Kim, Y. Deep stochastic logic gate networks. *IEEE Access*, 11:122488–122501, 2023.
- Kim, Y. Towards narrowing the generalization gap in deep boolean networks. *arXiv preprint arXiv:2409.05905*, 2024.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- Klement, E. P., Mesiar, R., and Pap, E. *Triangular Norms*. Springer, 2000.
- Komatsuzaki, A., Puigcerver, J., Lee-Thorp, J., Riquelme Ruiz, C., Mustafa, B., Ainslie, J., Tay, Y., Dehghani, M., and Houlisby, N. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *International Conference on Learning Representations*, 2023.
- Krizhevsky, A. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. GShard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations (ICLR)*, 2021.
- Liang, H., Zhang, S., Sun, J., He, X., Huang, W., Zhuang, K., and Li, Z. DARTS+: Improved differentiable architecture search with early stopping. *arXiv preprint arXiv:1909.06035*, 2019.
- Liu, H., Simonyan, K., and Yang, Y. DARTS: Differentiable architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- Liu, L., Dong, C., Liu, X., Yu, B., and Gao, J. Bridging discrete and backpropagation: Straight-through and beyond. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Maddison, C. J., Mnih, A., and Teh, Y. W. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations (ICLR)*, 2017.
- Mohamed, S., Rosca, M., Figurnov, M., and Mnih, A. Monte carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020.

- Petersen, F., Borgelt, C., Kuehne, H., and Deussen, O. Deep differentiable logic gate networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Puigcerver, J., Riquelme, C., Mustafa, B., and Houlsby, N. From sparse to soft mixtures of experts. In *International Conference on Learning Representations (ICLR)*, 2024.
- Razavi, A., van den Oord, A., and Vinyals, O. Generating diverse high-fidelity images with vq-vae-2. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Rudin, W. *Real and Complex Analysis*. McGraw-Hill, New York, 3 edition, 1987.
- Shah, R., Yan, M., Mozer, M. C., and Liu, D. Improving discrete optimisation via decoupled straight-through estimator. *arXiv preprint arXiv:2410.13331*, 2024.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.
- Van Den Oord, A., Vinyals, O., and Kavukcuoglu, K. Neural discrete representation learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- Van Krieken, E., Acar, E., and Van Harmelen, F. Analyzing differentiable fuzzy logic operators. *Artificial Intelligence*, 302:103602, 2022.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.
- Xie, S., Zheng, H., Liu, C., and Lin, L. SNAS: Stochastic neural architecture search. In *International Conference on Learning Representations (ICLR)*, 2019.
- Yousefi, S., Plesner, A., Aczel, T., and Wattenhofer, R. Mind the gap: Removing the discretization gap in differentiable logic gate networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Zela, A., Elsken, T., Saikia, T., Marrakchi, Y., Brox, T., and Hutter, F. Understanding and robustifying differentiable architecture search. In *ICLR*, 2020.
- Zhang, J. and Ding, Z. Small temperature is all you need for differentiable architecture search. In *The 27th Pacific-Asia Conference on Knowledge Discovery and Data Mining (PAKDD)*, pp. 303–315, 2023.
- Zhang, Z., Lin, Y., Liu, Z., Li, P., Sun, M., and Zhou, J. MoEfication: Transformer feed-forward layers are mixtures of experts. In *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 877–890, 2022.
- Zhou, Y., Lei, T., Liu, H., Du, N., Huang, Y., Zhao, V., Dai, A., Chen, Z., Le, Q., and Laudon, J. Mixture-of-experts with expert choice routing. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A. Potential Implications for Related Domains

Our theoretical framework may extend beyond logic gate networks. The forward alignment principle, which holds that training-time selection should match inference-time selection, predicts which methods avoid discretization gap. We briefly note potential connections; direct validation remains future work.

Neural Architecture Search. Differentiable NAS (Liu et al., 2019) relaxes discrete architecture selection via softmax-weighted operation mixtures, enabling gradient-based optimization. However, this introduces a training-inference gap: the soft supernet used during search differs fundamentally from the discrete architecture deployed at test time. This gap manifests as various pathologies, including skip connection collapse where parameter-free operations dominate (Chu et al., 2020; Zela et al., 2020), depth mismatch between search and evaluation (Chen et al., 2019), and overfitting of architecture parameters (Liang et al., 2019). Remedies include independent operation weighting (Chu et al., 2020), perturbation-based regularization (Zela et al., 2020), and Gumbel-softmax sampling (Xie et al., 2019; Dong & Yang, 2019). Our work addresses the analogous gap in logic gate networks, where the mismatch between real-valued relaxations and discrete Boolean operations causes similar performance degradation.

Mixture of Experts. Traditional MoE (Jacobs et al., 1991; Jordan & Jacobs, 1994) uses soft gating, but modern large-scale models have largely converged on hard routing. Switch Transformer (Fedus et al., 2022) uses hard top-1 (sparse) routing during both training and inference, avoiding soft-to-hard mismatch entirely. GShard (Lepikhin et al., 2021) and Expert Choice (Zhou et al., 2022) similarly employ hard routing with auxiliary losses for load balancing. Soft MoE (Puigcerver et al., 2024) uses soft routing in both phases to avoid mismatch while accepting the cost of activating all experts. Recent work enables transitions between dense and sparse architectures (Komatsuzaki et al., 2023; Zhang et al., 2022), converting pretrained dense models into MoE without retraining from scratch. Our framework provides one perspective on why hard routing dominates, though MoE faces distinct challenges (load balancing, expert utilization) orthogonal to selection gap.

Vector Quantization. VQ-VAE (Van Den Oord et al., 2017; Razavi et al., 2019) learns discrete representations via hard codebook selection (nearest neighbor lookup) with straight-through gradient estimation. The design was motivated by learning discrete latent structure and avoiding posterior collapse in models with powerful decoders. From our analysis perspective, this architecture naturally exhibits zero discretization gap: the forward pass uses identical hard selection during both training and inference. The commitment loss encourages encoder outputs to stay close to their assigned codebook entries, which can be interpreted as reducing the entropy of the implicit selection distribution, analogous to encouraging gate probabilities toward deterministic values in our framework.

B. Training-Inference Gap Across Domains

Table 3 summarizes the training-inference gap pattern across domains using discrete selection.

Table 3. Training-inference gap in discrete selection. Domains using soft training with hard inference suffer from mismatch; those using consistent selection avoid it.

Domain	Discrete Choices	Training	Inference	Gap?
Logic Gate Networks (Petersen et al., 2022; Kim, 2023; 2024)	Boolean gates	Soft mixture	Hard (argmax)	Yes
NAS (DARTS) (Liu et al., 2019)	Operations	Soft mixture	Hard (argmax)	Yes
NAS (SNAS/GDAS) (Xie et al., 2019; Dong & Yang, 2019)	Operations	Hard (sample)	Hard (argmax)	Small
MoE (traditional) (Jacobs et al., 1991)	Expert networks	Soft gating	Soft gating	No
MoE (sparse) (Fedus et al., 2022)	Expert networks	Hard (top-k)	Hard (top-k)	No
VQ-VAE (Van Den Oord et al., 2017)	Codebook vectors	Hard (nearest)	Hard (nearest)	No

C. Proofs of Theoretical Results

This appendix provides complete proofs for all theoretical results stated in Section 4. We use notation consistent with the main text: h^M denotes method M ’s forward output, g_i denotes gate i ’s output, and Δh_{sel} denotes the selection gap.

C.1. Proposition 1 (Gumbel-Max Theorem)

This is a classical result (Gumbel, 1954). We include a sketch for completeness.

Proof Sketch. Let $G_i \stackrel{\text{iid}}{\sim} \text{Gumbel}(0, 1)$ with CDF $F(x) = e^{-e^{-x}}$ and PDF $f(x) = e^{-x}e^{-e^{-x}}$. For option j to win, we need $G_i < z_j - z_i + G_j$ for all $i \neq j$.

Conditioning on $G_j = g$ and applying the CDF: $\mathbb{P}(G_i < z_j - z_i + g) = e^{-e^{-(z_j - z_i + g)}} = e^{-e^{-g}e^{z_i - z_j}}$.

By independence, the probability that j wins is the product over all $i \neq j$:

$$\mathbb{P}(\arg \max_i (z_i + G_i) = j \mid G_j = g) = \prod_{i \neq j} e^{-e^{-g}e^{z_i - z_j}} = \exp\left(-e^{-g} \sum_{i \neq j} e^{z_i - z_j}\right). \quad (12)$$

Marginalizing over G_j with PDF $f(g)$ and noting $1 + \sum_{i \neq j} e^{z_i - z_j} = \sum_i e^{z_i - z_j}$:

$$\mathbb{P}(\arg \max_i (z_i + G_i) = j) = \int_{-\infty}^{\infty} e^{-g} \exp\left(-e^{-g} \sum_i e^{z_i - z_j}\right) dg. \quad (13)$$

Substituting $u = e^{-g}$ gives $\int_0^{\infty} e^{-u} \sum_i e^{z_i - z_j} du = \frac{1}{\sum_i e^{z_i - z_j}} = \frac{e^{z_j}}{\sum_i e^{z_i}} = \text{softmax}(\mathbf{z})_j$.

Scale invariance: $\arg \max_i v_i = \arg \max_i (v_i/c)$ for any $c > 0$. $\square$

C.2. Proof of Corollary 2 (Gumbel-ST: Selection Gap)

Proof. Let $k = \arg \max_i (z_i + G_i)$ denote the stochastically selected gate. The training output is $h^M = g_k$.

By Proposition 1, $\mathbb{P}(k = i) = p_i = \text{softmax}(\mathbf{z})_i$. Taking expectations:

$$\mathbb{E}[h^M] = \sum_{i=0}^{K-1} p_i g_i. \quad (14)$$

The selection gap compares to g_{i^*} where $i^* = \arg \max_i z_i$:

$$\mathbb{E}[\Delta h_{\text{sel}}] = \sum_{i=0}^{K-1} p_i g_i - g_{i^*} = \sum_{i \neq i^*} p_i (g_i - g_{i^*}). \quad (15)$$

Since $p_i = \text{softmax}(\mathbf{z})_i$ is determined by raw logits (no temperature in the Gumbel-Max forward pass), the expected gap is independent of backward temperature τ_b . $\square$

C.3. Proof of Proposition 3 (Hard-ST: Zero Selection Gap)

Proof. Hard-ST uses $i^* = \arg \max_i z_i$ for both training and inference. Thus $h^M = g_{i^*} = h^{\text{hard}}$, yielding $\Delta h_{\text{sel}} = 0$ exactly. $\square$

C.4. Proof of Proposition 4 (Mixture Methods: Selection Gap)

Proof. Soft-Mix. The training output is $h^M = \sum_i \pi_i(\tau) g_i$ where $\pi_i(\tau) = \text{softmax}(\mathbf{z}/\tau)_i$. The selection gap:

$$\Delta h_{\text{sel}} = \sum_{i=0}^{K-1} \pi_i(\tau) g_i - g_{i^*} = \sum_{i \neq i^*} \pi_i(\tau) (g_i - g_{i^*}). \quad (16)$$

Since $\pi_i(\tau)$ depends explicitly on τ , the selection gap is temperature-dependent.

Soft-Gumbel. The training output is $h^M = \sum_i \tilde{\pi}_i(\tau) g_i$ where $\tilde{\pi}_i(\tau) = \text{softmax}((\mathbf{z} + \mathbf{G})/\tau)_i$. The selection gap has the same form:

$$\Delta h_{\text{sel}} = \sum_{i \neq i^*} \tilde{\pi}_i(\tau) (g_i - g_{i^*}). \quad (17)$$

Gumbel noise perturbs the weights but preserves the mixture structure, so the gap remains temperature-dependent. $\square$

C.5. Proof of Theorem 6 (Unified Selection Gap Bound)

Proof. Starting from the selection gap with effective weights $\mathbf{w}$:

$$|\Delta h_{\text{sel}}| = \left| \sum_{i \neq i^*} w_i (g_i - g_{i^*}) \right|. \quad (18)$$

Applying the triangle inequality and bounding $|g_i - g_{i^*}| \leq \Delta_{\max}$:

$$|\Delta h_{\text{sel}}| \leq \sum_{i \neq i^*} w_i |g_i - g_{i^*}| \leq \sum_{i \neq i^*} w_i \cdot \Delta_{\max} = (1 - w_{i^*}) \cdot \Delta_{\max}. \quad (19)$$

The bound is tight when all non-winner weight concentrates on the gate with maximum deviation. $\square$

C.6. Proof of Proposition 7 (Loss Gap Bound)

Proof. For GroupSum architecture, the class c output is $y_c = \sum_{n \in \mathcal{G}_c} h_n$. By linearity, the logit gap decomposes as $\Delta y_c = \sum_{n \in \mathcal{G}_c} \Delta h_n$.

By first-order Taylor expansion around $\mathbf{y}^{\text{hard}}$:

$$\mathcal{L}(\mathbf{y}^M) \approx \mathcal{L}(\mathbf{y}^{\text{hard}}) + \nabla_{\mathbf{y}} \mathcal{L} \big|_{\mathbf{y}^{\text{hard}}}^\top \Delta + O(\|\Delta\|^2), \quad (20)$$

where $\Delta = \mathbf{y}^M - \mathbf{y}^{\text{hard}}$ and $\Delta \mathcal{L} \equiv \mathcal{L}(\mathbf{y}^M) - \mathcal{L}(\mathbf{y}^{\text{hard}})$ denotes the loss gap.

Applying Hölder's inequality with $(p, q) = (1, \infty)$:

$$|\nabla_{\mathbf{y}} \mathcal{L}^\top \Delta| \leq \|\nabla_{\mathbf{y}} \mathcal{L}\|_1 \cdot \|\Delta\|_\infty. \quad (21)$$

For cross-entropy loss, let $\sigma_c = \text{softmax}(\mathbf{y}^{\text{hard}})_c$ denote the predicted probability for class c at inference. The gradient is:

$$\frac{\partial \mathcal{L}}{\partial y_c} = \sigma_c - \mathbf{1}[c = y]. \quad (22)$$

The L_1 norm (splitting the true class $c=y$ from the rest, with $\sigma_y \leq 1$ so $|\sigma_y - 1| = 1 - \sigma_y$, and the softmax normalization $\sum_c \sigma_c = 1$ so $\sum_{c \neq y} \sigma_c = 1 - \sigma_y$):

$$\|\nabla_{\mathbf{y}} \mathcal{L}\|_1 = \sum_c \left| \frac{\partial \mathcal{L}}{\partial y_c} \right| = |\sigma_y - 1| + \sum_{c \neq y} \sigma_c = (1 - \sigma_y) + (1 - \sigma_y) = 2(1 - \sigma_y). \quad (23)$$

For the logit gap, using the node-level bound (Theorem 6) with $\Delta_{\max} = 1$:

$$\|\Delta\|_\infty = \max_c |\Delta y_c| \leq \max_c \sum_{n \in \mathcal{G}_c} |\Delta h_n| \leq \max_c \sum_{n \in \mathcal{G}_c} (1 - w_{n, i_n^*}). \quad (24)$$

Combining:

$$|\Delta \mathcal{L}| \leq 2(1 - \sigma_y) \max_c \sum_{n \in \mathcal{G}_c} (1 - w_{n, i_n^*}) + O(\|\Delta\|^2). \quad (25)$$

This bound shows that loss gap is controlled by the product of prediction uncertainty $(1 - \sigma_y)$ and total non-commitment $\sum_n (1 - w_{n, i_n^*})$. When nodes fully commit, the bound vanishes. For Hard-ST, selection gap is exactly zero at every node, so the bound is trivially satisfied. $\square$

C.7. Proof of Proposition 8 (Gradient Structure)

Proof. For any method using surrogate weights $w_j(\tau)$, the gradient with respect to logit z_j is:

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\partial \mathcal{L}}{\partial h} \cdot \frac{\partial h}{\partial z_j}. \quad (26)$$

The surrogate gradient (straight-through for hard methods, direct for soft methods) is obtained in two steps, separating the single approximation from the exact algebra.

Step 1 (STE approximation). The non-differentiable argmax is replaced by the soft surrogate $\bar{h} = \sum_i w_i(\tau)g_i$ (Bengio et al., 2013), so that

$$\frac{\partial h}{\partial z_j} \approx \frac{\partial \bar{h}}{\partial z_j} = \frac{\partial}{\partial z_j} \sum_i w_i(\tau)g_i. \quad (\text{STE substitution}) \quad (27)$$

Step 2 (exact, given Step 1). Applying the chain rule to the soft surrogate is an exact equality:

$$\frac{\partial \bar{h}}{\partial z_j} = \sum_i g_i \frac{\partial w_i}{\partial z_j}. \quad (28)$$

The approximation symbol thus belongs solely to the STE substitution in Step 1; all subsequent algebra is exact. This substitution is a design choice shared by every straight-through method, not a gap specific to this derivation.

For softmax weights $w_i = \text{softmax}(\mathbf{z}/\tau_b)_i$, the Jacobian is $\frac{\partial w_i}{\partial z_j} = \frac{1}{\tau_b} w_i(\mathbf{1}[i=j] - w_j)$. Substituting this into the Step-2 expression and using $\bar{h} = \sum_i w_i g_i$,

$$\frac{\partial \bar{h}}{\partial z_j} = \frac{1}{\tau_b} \sum_i g_i w_i (\mathbf{1}[i=j] - w_j) = \frac{1}{\tau_b} (g_j w_j - w_j \sum_i g_i w_i) = \frac{1}{\tau_b} w_j (g_j - \bar{h}). \quad (29)$$

Multiplying by $\delta = \partial \mathcal{L} / \partial h$ (the chain rule) then gives

$$\frac{\partial \mathcal{L}}{\partial z_j} = \frac{\delta}{\tau_b} \cdot w_j(\tau_b) \cdot (g_j - \bar{h}), \quad (30)$$

where $\bar{h} = \sum_i w_i g_i$. (For mixture methods, the forward and backward temperatures coincide, $\tau_b = \tau$.) $\square$

Where δ is evaluated. The gradient formula is identical across methods, but δ is evaluated at different points. **Mixture methods:** $\delta = \partial \mathcal{L} / \partial h|_{h=\bar{h}}$; the gradient exactly optimizes $\mathcal{L}(\bar{h}, y)$. **Hard-ST:** $\delta = \partial \mathcal{L} / \partial h|_{h=g_{i^*}}$; forward uses discrete output, backward uses soft surrogate. **Gumbel-ST:** $\delta = \partial \mathcal{L} / \partial h|_{h=g_k}$ for sampled k ; in expectation, approximates $\nabla_{\mathbf{z}} \sum_i p_i \mathcal{L}(g_i, y)$.

Competition term. The term $(g_j - \bar{h})$ induces competition: gates with $g_j > \bar{h}$ (outperforming the mixture) receive positive gradient when $\delta < 0$, increasing their selection weight. This drives commitment over training.

C.8. Analysis of Proposition 9 (Competitive Gate Selection)

We analyze how the competition term $(g_j - \bar{h})$ leads to different behaviors across methods.

Output spaces. The achievable output space differs by method. **Mixture methods:** $\mathcal{H}_{\text{soft}} = \{\sum_i w_i g_i : \mathbf{w} \in \Delta^{K-1}\} = [0, 1]$, the convex hull of gate outputs. **Hard methods:** $\mathcal{H}_{\text{hard}} = \{g_0, \dots, g_{K-1}\} \subseteq \{0, 1\}$, the discrete set of gate outputs.

Mixture methods can hedge. Since $\mathcal{H}_{\text{soft}} = [0, 1]$ is continuous, multiple weight configurations can achieve similar blended outputs. The optimizer can find favorable combinations where $\bar{h} = \sum_i w_i g_i$ reduces loss, even if no single gate g_i performs well individually. This is hedging: relying on the blend rather than committing to one gate.

Hard methods must commit. Since $\mathcal{H}_{\text{hard}} \subseteq \{0, 1\}$ contains only discrete outputs, the optimizer must find a single gate that performs well. For Hard-ST, this requires $z_k > z_j$ for all $j \neq k$. For Gumbel-ST, reliable selection requires $p_k \approx 1$, meaning $z_k \gg z_j$. Both conditions demand strong logit separation, forcing commitment.

Connection to selection gap. Hedging solutions have $\bar{h} \notin \{0, 1\}$, so $\bar{h} \neq g_{i^*}$ generically, producing selection gap. Commitment solutions have concentrated weights, so $\bar{h} \approx g_{i^*}$, eliminating gap.

C.9. Proof of Proposition 10 (Convergence Scaling)

Proof. Consider a single selection node choosing among K options with logits $\mathbf{z} = (z_1, \dots, z_K)$, learning rate η , and backward temperature τ_b .

Step 1: Softmax gradient scaling. The softmax probabilities with temperature are:

$$p_i = \frac{e^{z_i/\tau_b}}{\sum_j e^{z_j/\tau_b}}. \quad (31)$$

The gradient of p_i with respect to z_i is:

$$\frac{\partial p_i}{\partial z_i} = \frac{1}{\tau_b} p_i (1 - p_i) = O(1/\tau_b). \quad (32)$$

Thus, gradient magnitude scales inversely with τ_b .

Step 2: Required logit gap for target confidence. To achieve confidence $p_{i^*} \geq 1 - \epsilon$ for the dominant option i^* , we require:

$$\frac{e^{z_{i^*}}}{\sum_j e^{z_j}} \geq 1 - \epsilon. \quad (33)$$

Assuming other logits are approximately equal to some baseline z_0 , let $\Delta = z_{i^*} - z_0$. Then:

$$\frac{e^\Delta}{e^\Delta + (K - 1)} \geq 1 - \epsilon \quad (34)$$

$$e^\Delta \geq (1 - \epsilon)(e^\Delta + (K - 1)) \quad (35)$$

$$\epsilon \cdot e^\Delta \geq (1 - \epsilon)(K - 1) \quad (36)$$

$$\Delta \geq \log \left(\frac{(1 - \epsilon)(K - 1)}{\epsilon} \right). \quad (37)$$

For small ϵ and fixed K :

$$\Delta = O(\log(1/\epsilon)). \quad (38)$$

Step 3: Per-step logit update. With learning rate η , gradient magnitude $O(1/\tau_b)$, and approximately constant upstream gradient u , the per-step update to the dominant logit is:

$$\Delta z_{i^*} = \eta \cdot \frac{u}{\tau_b} \cdot p_{i^*} (1 - p_{i^*}) = O(\eta/\tau_b). \quad (39)$$

Similarly, non-dominant logits decrease, so the gap Δ grows at rate $O(\eta/\tau_b)$ per step.

Step 4: Convergence time. The number of steps to achieve the required gap is:

$$T(\tau_b) = \frac{\text{Required gap}}{\text{Gap growth per step}} = \frac{O(\log(1/\epsilon))}{O(\eta/\tau_b)} = O \left(\frac{\tau_b \cdot \log(1/\epsilon)}{\eta} \right). \quad (40)$$

□

D. CAGE Algorithm Details

A pseudocode of the CAGE is shown in Algorithm 1.

Algorithm 1 CAGE: Confidence-Adaptive Gradient Estimation

Require: $\tau_{\min} = 0.5, \tau_{\max} = 3.0, \beta = 0.99$
 Initialize $c_{\text{ema}} \leftarrow 1/K$
for each training step t **do**
 $c(t) \leftarrow \frac{1}{N} \sum_{n=1}^N \max_i \text{softmax}(\mathbf{z}_n)_i$
 $c_{\text{ema}} \leftarrow \beta \cdot c_{\text{ema}} + (1 - \beta) \cdot c(t)$
 $\tau_b \leftarrow \tau_{\max} - (\tau_{\max} - \tau_{\min}) \cdot c_{\text{ema}}$
 Forward: hard selection (Hard-ST or Gumbel-ST)
 Backward: ST gradients with temperature τ_b
end for

E. Critique of Hessian Regularization Hypothesis

Recent work by Yousefi et al. (2025) analyzed the discretization gap through the lens of “implicit Hessian regularization.” Their analysis assumes the loss $\mathcal{L}$ is computed directly on softmax outputs, yielding an expansion:

$$\begin{aligned}
 J(\mathbf{z}) &= \mathbb{E}_{\mathbf{G}}[\mathcal{L}(\text{softmax}((\mathbf{z} + \mathbf{G})/\tau))] \\
 &= \mathcal{L}(\text{softmax}(\mathbf{z}/\tau)) + \frac{\pi^2}{12\tau^2} \text{tr}(H_{\mathcal{L}}) + O(\tau^{-3}),
 \end{aligned} \tag{41}$$

where the Hessian trace term provides implicit regularization. The hypothesis is that Gumbel noise reduces mismatch through this regularization effect.

We identify three issues:

Mixture vs. Hard Forward. The analysis assumes mixture forward. For Gumbel-ST, the forward output is discrete ($h^M = g_k$), and the Hessian expansion does not directly apply.

Architecture Mismatch. In deep networks, selection logits propagate through multiple layers before reaching the loss. The relationship between selection temperature and loss Hessian is more complex than the single-layer analysis suggests.

Empirical Evidence. Soft-Gumbel and Gumbel-ST use identical Gumbel noise but exhibit markedly different gaps. This suggests gap reduction stems from forward alignment, not Hessian regularization.

F. Extended Gradient Analysis

Gumbel Methods Maintain Gradient Flow. Even at small τ_b , Gumbel noise can shift the winner: $k = \arg \max_i (z_i + G_i)$ differs from $i^* = \arg \max_i z_i$ with probability $1 - p_{i^*} > 0$.

When $k \neq i^*$, the output $h^M = g_k \neq g_{i^*}$ provides non-zero gradient signal. Over multiple training steps, different gates win, maintaining gradient flow to all gates. This exploration mechanism is distinct from the soft surrogate used in the backward pass.

Low-Temperature Failure Mode. For mixture methods at low τ , gradients vanish for two reasons: (1) the winner’s deviation $(g_{i^*} - \bar{h}) \rightarrow 0$ as $\bar{h} \rightarrow g_{i^*}$, and (2) non-winner weights $w_j \rightarrow 0$. Hard selection methods avoid this because δ is evaluated at the discrete output, not the collapsing mixture.

G. GroupSum Temperature Derivation

We derive the readout temperature T_{GS} of the GroupSum layer (Petersen et al., 2022; Kim, 2023) (distinct from the gate-selection temperature τ used elsewhere) that produces well-conditioned softmax outputs at initialization.

Consider classification with C classes, m neurons per class, each output $h_n \in [0, 1]$, GroupSum readout $s_c = \sum_{n \in \mathcal{G}_c} h_n$, and scaled logits $s'_c = s_c/T_{\text{GS}}$.

Statistics at Initialization. With random gate selection, $h_n \sim \text{Bernoulli}(0.5)$, giving $\mathbb{E}[s_c] = 0.5m$ and $\text{Var}[s_c] = 0.25m$. Since softmax is translation-invariant, only logit differences matter. Let $\Delta s = s_i - s_j$ denote the difference between two

class sums. Since s_i and s_j are independent:

$$\text{Var}[\Delta s] = \text{Var}[s_i] + \text{Var}[s_j] = 0.5m, \quad \text{Std}[\Delta s] = \sqrt{0.5}\sqrt{m} \approx 0.71\sqrt{m}. \quad (42)$$

Optimal Temperature. We target $p_{\max} \approx 0.66$ (a design choice balancing gradient flow: not too peaked, not too uniform). Solving $p_{\max} = \exp(\Delta s/T_{\text{GS}})/[\exp(\Delta s/T_{\text{GS}}) + (C - 1)] = 0.66$ gives $\Delta s/T_{\text{GS}} \approx \ln(C - 1) + 0.7$. Using two standard deviations as typical maximum difference ($\Delta s \approx 1.42\sqrt{m}$):

$$T_{\text{GS}} = \frac{1.42\sqrt{m}}{\ln(C - 1) + 0.7} = \alpha(C)\sqrt{m}, \quad (43)$$

where $\alpha(C) = 1.42/(\ln(C - 1) + 0.7)$. For common class counts: $\alpha(10) = 0.49$, $\alpha(100) = 0.27$, $\alpha(1000) = 0.19$.

H. Experimental Details

H.1. Hyperparameters

Table 4 summarizes the training configurations used in our experiments. Each experimental run (one method, temperature, and seed) was executed on a single NVIDIA H100 GPU.

Table 4. Training configuration.

Parameter	MNIST	CIFAR-10
Neurons per layer	64,000	128,000
Layers	4, 5, 6	4, 5, 6
Gates per node	16	16
Batch size	512	512
Learning rate	0.01	0.05
Optimizer	Adam	Adam
Iterations	50,000	60,000
Seeds	3	3
<i>Temperature sweep:</i> $\tau \in \{0.05, 0.1, 0.5, 1.0, 2.0\}$		
<i>CAGE:</i> $\tau_{\max} = 3.0$, $\tau_{\min} = 0.5$, $\beta = 0.99$		

H.2. Cross-Dataset Summary

Table 5 provides a consolidated view of key metrics across all datasets at $L=6$, the primary configuration reported in the main text.

Hard-ST achieves zero selection gap on MNIST-Binary and CIFAR-10-Binary (binary inputs eliminate computation gap). On MNIST, the small 0.4% gap is purely computation gap from continuous inputs. Gumbel-ST fails catastrophically at low τ across all datasets, with accuracy variation of 39–47 percentage points. CAGE eliminates temperature sensitivity, reducing accuracy variation to <1 percentage point for both Hard-ST and Gumbel-ST.

H.3. Gap Dynamics

MNIST. Table 6 shows total gap dynamics (selection + computation) during training on MNIST. The format “Final [min, max]” indicates the final gap at convergence and the range observed during the last 80% of training (excluding initial warm-up).

MNIST has continuous pixel values in $[0, 1]$, so the measured gap includes both selection and computation components. Hard-ST shows a $\sim 0.2\%$ final gap ($\sim 0.4\%$ peak) from computation gap alone (selection gap is zero by construction). Gumbel-ST exhibits catastrophic negative gaps at $\tau \leq 0.1$, indicating training failure where deployment accuracy (A^{hard}) exceeds A^M .

MNIST-Binary. Table 7 shows gap dynamics on MNIST-Binary. With binary inputs, computation gap is zero, so this table isolates pure selection gap.

Table 5. Cross-dataset summary ($L=6$). Peak selection gap (%) during training and accuracy range (%) across $\tau \in \{0.05, 0.1, 0.5, 1.0, 2.0\}$. Hard-ST achieves zero selection gap and minimal accuracy range across all datasets. Gumbel-ST suffers catastrophic training failure at low τ . **Bold**: zero selection gap; underline: best accuracy range.

Method	MNIST		MNIST-Binary		CIFAR-10-Binary	
	Peak Gap	Acc Range	Peak Gap	Acc Range	Peak Gap	Acc Range
Soft-Mix	23%	1.4%	24%	1.1%	9%	7.5%
Soft-Gumbel	40%	12.0%	40%	12.2%	3.5%	5.4%
Hard-ST	0.4%*	1.3%	0%	1.1%	0%	7.2%
Gumbel-ST	62% [†]	46.3%	61% [†]	47.3%	16% [†]	39.5%
Hard-ST+CAGE	0.3%*	<u>0.1%</u>	0%	<u>0.0%</u>	0%	<u>0.5%</u>
Gumbel-ST+CAGE	9%	<u>0.1%</u>	9%	<u>0.1%</u>	0.7%	<u>0.6%</u>

* Includes computation gap from continuous inputs (selection gap is zero). [†] Training failure at low τ produces large negative gap during training.

Table 6. Total gap (selection + computation) trajectory on MNIST. Format: Final [min, max] (%). Range computed over last 80% of training. On continuous MNIST the computation gap is nonzero, so Hard-ST’s stable near-zero gap ($\sim 0.2\%$) reflects computation gap alone (its selection gap is exactly zero); Soft-Gumbel shows large transient peaks that increase with τ .

Method	L	$\tau=0.05$	$\tau=0.1$	$\tau=0.5$	$\tau=1.0$	$\tau=2.0$
Soft-Mix	4	+0.3 [+0.2,+0.4]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.1 [+0.0,+0.3]	+0.2 [+0.2,+0.5]
	5	+0.3 [+0.2,+0.4]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.1 [+0.1,+0.3]	+0.3 [+0.2,+1.5]
	6	+0.3 [+0.1,+0.4]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.1 [+0.1,+0.3]	+0.7 [+0,+23]
Soft-Gumbel	4	+0.6 [-31,+2]	+0.2 [-15,+2]	+0.3 [-0.1,+2.3]	+0.3 [+0,+10]	+0.5 [+0,+21]
	5	-0.2 [-33,+1]	+0.2 [-33,+2]	+0.3 [+0.0,+2.9]	+0.4 [+0,+16]	+1.1 [+1,+33]
	6	+0.2 [-29,+4]	+0.6 [-25,+4]	+0.4 [-0,+9]	+0.4 [+0,+16]	+1.7 [+1,+40]
Hard-ST	4	+0.2 [+0.2,+0.4]	+0.2 [+0.2,+0.3]	+0.3 [+0.2,+0.4]	+0.2 [+0.1,+0.4]	+0.2 [+0.1,+0.4]
	5	+0.3 [+0.2,+0.4]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]
	6	+0.2 [+0.2,+0.4]	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.2]	+0.2 [+0.1,+0.4]	+0.1 [+0.0,+0.3]
Gumbel-ST	4	-46.3 [-48,-41]	-43.2 [-56,-42]	+0.2 [-0.2,+0.4]	+0.1 [+0.0,+0.6]	+0.1 [+0.0,+0.8]
	5	-41.8 [-43,-28]	-55.2 [-59.0,-54.5]	+0.1 [-0.4,+0.2]	+0.2 [-0.1,+0.4]	+0.2 [+0.0,+0.9]
	6	-41.8 [-43,-31]	-58.2 [-62,-55]	+0.2 [-0.8,+0.3]	+0.2 [-0.1,+0.4]	+0.2 [+0.0,+1.4]
Hard-ST+CAGE	4	+0.1 [+0.0,+0.3]	+0.2 [+0.0,+0.3]	+0.2 [+0.0,+0.3]	+0.2 [+0.1,+0.3]	+0.3 [+0.1,+0.3]
	5	+0.2 [+0.1,+0.3]	+0.2 [+0.1,+0.3]	+0.2 [+0.0,+0.3]	+0.2 [+0.1,+0.3]	+0.1 [+0.0,+0.3]
	6	+0.2 [+0.0,+0.3]	+0.2 [+0.0,+0.3]	+0.1 [+0.0,+0.2]	+0.2 [+0.1,+0.3]	+0.1 [+0.0,+0.3]
Gumbel-ST+CAGE	4	+0.2 [+0.0,+0.5]	+0.3 [+0.0,+0.7]	+0.2 [-0.1,+1.1]	+0.2 [+0.0,+0.5]	+0.2 [+0.0,+0.6]
	5	+0.2 [+0.0,+0.9]	+0.1 [+0.0,+1.3]	+0.2 [+0.0,+1.8]	+0.2 [-0.1,+0.8]	+0.1 [+0.0,+0.9]
	6	+0.2 [-0.1,+3.2]	+0.2 [-0,+5]	+0.2 [-0,+5]	+0.2 [-0.1,+2.6]	+0.1 [-0.2,+1.7]

Hard-ST and Hard-ST+CAGE achieve exactly 0% gap at all temperatures and layers. The cells show “+0.0” with no range because the gap is identically zero throughout training. This is the strongest validation of forward alignment: when training uses the same argmax selection as inference, there is no selection mismatch by construction.

CIFAR-10-Binary. Table 8 shows gap dynamics on CIFAR-10-Binary binarized with 31 evenly-spaced thresholds per channel.

CIFAR-10-Binary shows smaller transient gaps for Soft-Gumbel compared to MNIST (max $\sim 3.5\%$ vs $\sim 40\%$). This may be because the harder classification task requires more commitment to specific gates, reducing the benefit of hedging. Gumbel-ST still fails at low τ but less catastrophically (-14% vs -59%).

H.4. Heatmap Visualization

Figure 4 visualizes gap across the full (layer $\times$ temperature) grid, providing an overview of all experimental conditions.

H.5. Statistical Tests

We report statistical significance tests for the key claims in this paper. All tests use paired t -tests with $n = 3$ seeds per configuration. With small sample sizes and low variance, even small differences can achieve statistical significance. We therefore distinguish between **statistical significance** ($p < 0.05$) and **practical significance** ($|\Delta \text{Accuracy}| > 5\%$ or $|\Delta \text{Gap}| > 5\%$).

Table 7. Gap trajectory on MNIST-Binary. Format: Final [min, max] (%). With binary inputs, computation gap = 0, so this isolates pure selection gap. Hard-ST achieves exactly 0% at all temperatures.

Method	L	$\tau=0.05$	$\tau=0.1$	$\tau=0.5$	$\tau=1.0$	$\tau=2.0$
Soft-Mix	4	+0.0 [-0.1,+0.0]	+0.0	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.2]	+0.0 [-0.1,+0.3]
	5	+0.0	+0.0	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+1.6]
	6	+0.0	+0.0	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.6 [+0,+24]
Soft-Gumbel	4	+0.3 [-31,+1]	+0.1 [-14,+1]	+0.1 [-0.3,+1.8]	+0.2 [+0,+9]	+0.2 [+0,+23]
	5	-0.1 [-35,+1]	-0.5 [-31,+2]	+0.2 [-0.4,+4.3]	+0.2 [+0,+18]	+0.7 [+0,+35]
	6	+0.7 [-30,+2]	+0.2 [-25,+2]	+0.2 [-1,+7]	+0.2 [+0,+17]	+2.1 [+2,+40]
Hard-ST	4	+0.0	+0.0	+0.0	+0.0	+0.0
	5	+0.0	+0.0	+0.0	+0.0	+0.0
	6	+0.0	+0.0	+0.0	+0.0	+0.0
Gumbel-ST	4	-45.1 [-47,-41]	-44.7 [-55,-43]	+0.0 [-0.5,+0.1]	+0.0 [-0.2,+0.3]	+0.0 [-0.1,+0.5]
	5	-38.7 [-39,-28]	-57.9 [-59.9,-55.3]	+0.0 [-0.7,+0.1]	+0.0 [-0.2,+0.3]	+0.0 [-0.2,+0.7]
	6	-40.4 [-41,-29]	-58.8 [-61,-55]	-0.1 [-1.3,+0.1]	+0.0 [-0.3,+0.3]	+0.0 [-0.3,+1.1]
Hard-ST+CAGE	4	+0.0	+0.0	+0.0	+0.0	+0.0
	5	+0.0	+0.0	+0.0	+0.0	+0.0
	6	+0.0	+0.0	+0.0	+0.0	+0.0
Gumbel-ST+CAGE	4	+0.1 [-0.2,+0.6]	+0.0 [-0.2,+0.8]	-0.1 [-0.3,+0.7]	+0.0 [-0.1,+0.4]	+0.0 [-0.2,+0.4]
	5	+0.0 [-0.2,+1.7]	+0.0 [-0.2,+2.3]	+0.0 [-0.2,+2.0]	+0.0 [-0.2,+0.8]	+0.0 [-0.2,+0.7]
	6	+0.0 [-0.3,+4.5]	+0.1 [-0,+9]	+0.0 [-0.2,+3.5]	+0.0 [-0.2,+1.6]	+0.0 [-0.2,+2.2]

Table 8. Gap trajectory on CIFAR-10-Binary. Format: Final [min, max] (%). Hard-ST achieves exactly 0% gap; Gumbel-ST fails at low τ but less severely than on MNIST.

Method	L	$\tau=0.05$	$\tau=0.1$	$\tau=0.5$	$\tau=1.0$	$\tau=2.0$
Soft-Mix	4	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.3,+0.3]	+0.1 [-0.3,+0.5]
	5	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.3,+0.3]	-0.1 [-0.2,+1.5]
	6	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.1,+0.1]	+0.0 [-0.2,+0.3]	+0.3 [-0,+9]
Soft-Gumbel	4	-0.2 [-2.3,+0.4]	+0.0 [-2.0,+0.4]	+0.0 [-0.7,+0.5]	+0.0 [-0.3,+0.3]	+0.1 [-0.2,+0.4]
	5	+0.0 [-2.5,+0.3]	-0.2 [-2.6,+0.4]	+0.0 [-0.8,+0.7]	+0.0 [-0.3,+0.5]	+0.1 [-0.3,+0.4]
	6	-0.3 [-3.5,+0.2]	+0.2 [-2.3,+0.4]	+0.0 [-0.7,+0.5]	+0.0 [-0.3,+0.5]	+0.0 [-0.3,+0.4]
Hard-ST	4	+0.0	+0.0	+0.0	+0.0	+0.0
	5	+0.0	+0.0	+0.0	+0.0	+0.0
	6	+0.0	+0.0	+0.0	+0.0	+0.0
Gumbel-ST	4	-9.6 [-11,-4]	-9.5 [-11.8,-8.2]	-0.2 [-0.8,+0.5]	+0.0 [-0.3,+0.2]	+0.0 [-0.4,+0.5]
	5	-5.2 [-8,-2]	-11.4 [-14.3,-10.4]	+0.0 [-0.9,+0.2]	+0.0 [-0.4,+0.3]	-0.1 [-0.4,+0.3]
	6	-7.5 [-8,-0]	-13.7 [-16.4,-12.6]	-0.4 [-1.2,+0.2]	+0.0 [-0.4,+0.3]	-0.1 [-0.3,+0.4]
Hard-ST+CAGE	4	+0.0	+0.0	+0.0	+0.0	+0.0
	5	+0.0	+0.0	+0.0	+0.0	+0.0
	6	+0.0	+0.0	+0.0	+0.0	+0.0
Gumbel-ST+CAGE	4	-0.1 [-0.6,+0.3]	-0.1 [-0.6,+0.3]	-0.1 [-0.6,+0.3]	-0.1 [-0.4,+0.4]	+0.0 [-0.6,+0.3]
	5	+0.0 [-0.6,+0.3]	-0.2 [-0.6,+0.3]	-0.1 [-0.6,+0.3]	+0.0 [-0.8,+0.3]	-0.2 [-0.5,+0.3]
	6	+0.0 [-0.6,+0.4]	+0.0 [-0.7,+0.4]	-0.1 [-0.7,+0.3]	+0.0 [-0.5,+0.3]	-0.1 [-0.5,+0.4]

Table 9 reports results for MNIST-Binary with $L=6$ layers, which isolates selection gap (computation gap is zero for binary inputs).

Findings. Gumbel-ST failure is both statistically and practically significant. The accuracy drop from $\tau = 2.0$ to $\tau = 0.05$ is highly significant ($t = -36.16$, $p < 0.001$, $d = 31.21$) and represents catastrophic training failure (47 percentage points accuracy loss).

Hard-ST shows statistical but not practical sensitivity to τ . While the accuracy difference is statistically significant ($p = 0.005$) due to low variance across seeds, the 1.1 percentage point range is negligible compared to Gumbel-ST’s 47 percentage point range. The claim of “temperature robustness” refers to practical robustness.

CAGE effectively resolves Gumbel-ST’s failure. At $\tau = 0.05$, CAGE improves Gumbel-ST accuracy by 47 percentage points ($t = -38.81$, $p < 0.001$, $d = 31.28$), restoring performance to the level achieved at higher temperatures.

Hard-ST achieves zero selection gap. The gap difference between Hard-ST (0%) and Soft-Gumbel (40% peak) at $\tau = 2.0$ is statistically significant ($p = 0.024$) with large effect size ($d = 5.15$).

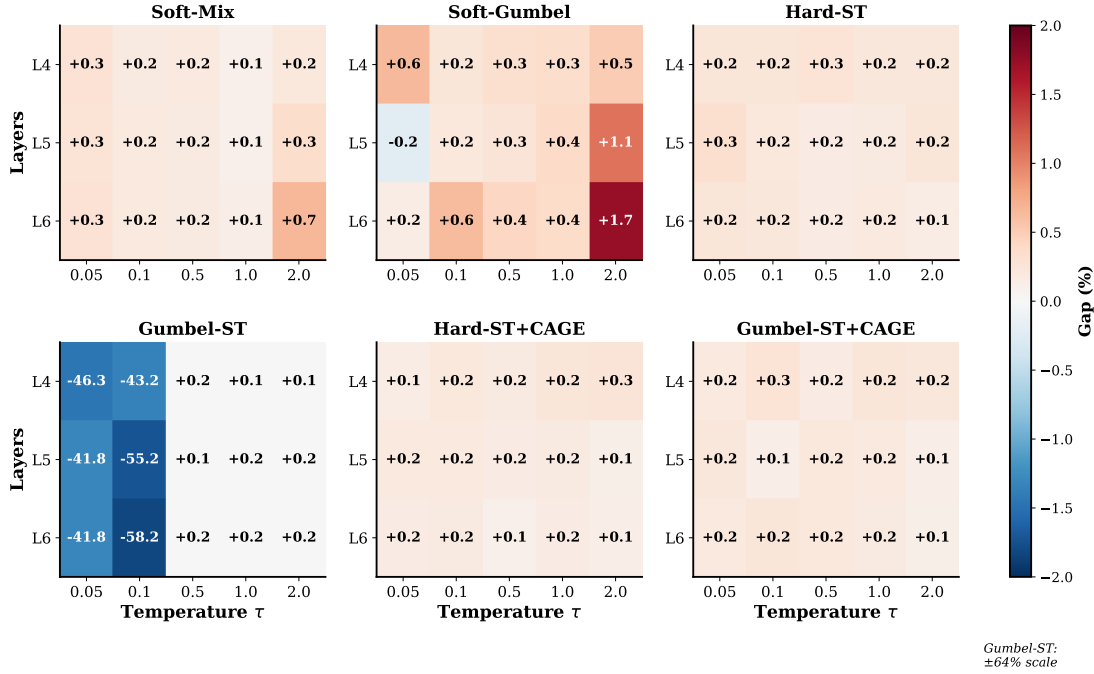


Figure 4. Gap heatmaps on MNIST. Hard-ST shows uniform near-zero gap (light colors) across all conditions. Gumbel-ST exhibits catastrophic negative gap at low τ (dark blue = training failure where deployment outperforms training). CAGE eliminates this failure, restoring uniform small gap.

Forward structure effect shows practical significance. The comparison between Soft-Gumbel and Gumbel-ST gaps (same Gumbel noise, different forward pass) shows a clear practical difference (40% vs 1% gap) with marginal statistical significance ($p = 0.093$, $d = 2.61$). The elevated p -value reflects high variance in Soft-Gumbel’s gap across seeds; the direction consistently supports the forward alignment hypothesis.

Table 9. Statistical significance of key claims (MNIST-Binary, $L=6$, $n=3$ seeds). Effect size d is Cohen’s d (Cohen, 1988).

Claim	Comparison	t	p	d	Stat.	Prac.
<i>Temperature sensitivity (accuracy)</i>						
Gumbel-ST fails at low τ	$\tau=0.05$ vs $\tau=2.0$	-36.16	<.001	31.21	***	✓
Hard-ST is τ -robust	$\tau=0.05$ vs $\tau=2.0$	-14.53	.005	16.26	**	–
<i>CAGE effectiveness</i>						
CAGE fixes Gumbel-ST	Gumbel-ST vs +CAGE ($\tau=0.05$)	-38.81	<.001	31.28	***	✓
<i>Selection gap comparisons (at $\tau=2.0$)</i>						
Hard-ST has zero selection gap	Hard-ST vs Soft-Gumbel	-6.30	.024	5.15	*	✓
<i>Forward vs backward structure</i>						
Forward type determines gap	Soft-Gumbel vs Gumbel-ST gap	3.05	.093	2.61	ns	✓

Significance: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns = not significant. Practical: ✓ = $|\Delta| > 5\%$ accuracy difference.

Cross-dataset consistency. Table 10 confirms these findings hold across datasets.

H.6. Gate Usage Analysis

Figure 5 shows the distribution of selected gates across all nodes in the trained networks.

Table 10. Cross-dataset verification of key claims ($L=6$, significance level).

Claim	MNIST	MNIST-Binary	CIFAR-10-Binary
Gumbel-ST fails at low τ	***	***	***
Hard-ST is τ -robust	**†	**†	***†
CAGE fixes Gumbel-ST	***	***	***

Significance: *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

†Statistically significant but practically robust: MNIST datasets show $<2\%$ accuracy range, CIFAR-10-Binary shows $\sim 6\%$ range (all far below Gumbel-ST’s $>39\%$).

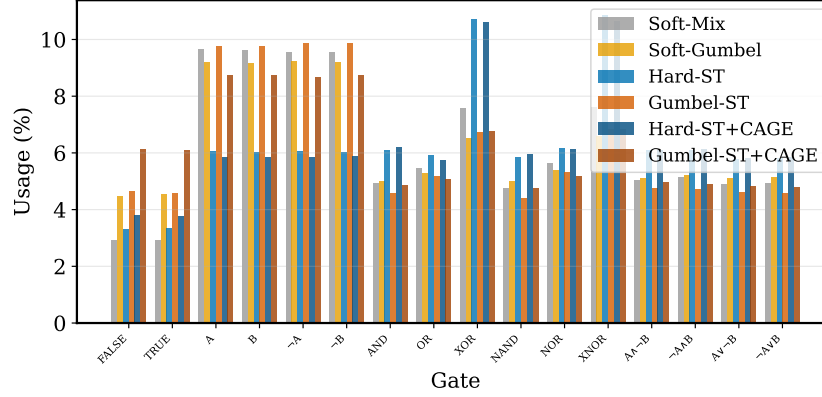


Figure 5. Gate usage distribution (MNIST-Binary, $L=6$, $\tau=1.0$). Hard-ST methods show pronounced XOR/XNOR preference ($\sim 11\%$ each vs. $\sim 6\%$ expected under uniform selection), while soft methods distribute usage more uniformly. This suggests hard selection encourages commitment to more expressive Boolean operations.

I. CAGE Ablations

This appendix isolates two questions about CAGE: (i) does the *direction* of the confidence-to-temperature schedule matter, or does merely avoiding low temperatures suffice; and (ii) how sensitive is CAGE to its own bound hyperparameters ($\tau_{\max}, \tau_{\min}$). All runs use Gumbel-ST on MNIST-Binary with $K=64k$ neurons per layer, $L=6$, and 3 seeds. **Last-10** denotes the average of the last 10 recorded test accuracies, our primary metric for terminal convergence stability; **Converge** is the number of iterations to first reach 90% of the final accuracy.

I.1. Reverse-CAGE: the Confidence Direction Is Essential

To test whether the failure at low temperature reflects low temperature *per se* or low temperature *without an escape mechanism*, we compare CAGE against **Reverse-CAGE**, which inverts the schedule: it starts at $\tau_{\min}$ and *increases* toward $\tau_{\max}$ as confidence rises. We sweep the active bound across three temperature scales (Table 11).

Table 11. CAGE vs. Reverse-CAGE (Gumbel-ST, MNIST-Binary, $K=64k$, $L=6$, 3 seeds; Last-10 average). “Converge” is iterations to reach 90% of final accuracy; $\times$ denotes non-convergence. τ_b standard deviation is < 0.005 across seeds (omitted).

Scheduler	Last-10 (%)	Gap (%)	Converge	τ_b @1k	τ_b final
CAGE ($\tau_{\max}=0.05$)	38.0 ± 9.2	-39.7 ± 4.1	$\times$	0.048	0.046
Rev-CAGE ($\tau_{\min}=0.05$)	98.1 ± 0.0	$+0.05 \pm 0.12$	$8,000 \pm 0$	0.214	2.18
CAGE ($\tau_{\max}=0.1$)	75.3 ± 1.7	-65.8 ± 0.5	$\times$	0.094	0.081
Rev-CAGE ($\tau_{\min}=0.1$)	98.1 ± 0.0	-0.03 ± 0.10	$6,667 \pm 471$	0.280	2.21
CAGE ($\tau_{\max}=0.5$)	94.9 ± 0.1	-0.60 ± 0.10	$3,000 \pm 0$	0.440	0.197
Rev-CAGE ($\tau_{\min}=0.5$)	98.1 ± 0.0	$+0.05 \pm 0.02$	$3,333 \pm 471$	0.731	2.33

The τ_b trajectory reveals the mechanism. CAGE with a low ceiling ($\tau_{\max}=0.05$) is permanently trapped: τ_b barely moves ($0.048 \rightarrow 0.046$) and accuracy collapses to 38.0% (± 9.2). The failure is a self-reinforcing loop: low τ concentrates gradient on a random early winner, which prevents confidence growth, which in turn keeps τ_b pinned at the low ceiling.

Reverse-CAGE with the same starting value ($\tau_{\min}=0.05$) breaks the loop: although τ_b begins at 0.05, it rises as soon as any learning occurs ($0.05 \rightarrow 0.21$ by iteration $1k \rightarrow 2.18$ at convergence), reaching 98.1%, identical to standard CAGE.

Convergence speed depends directly on the starting τ_b : Rev-CAGE(0.5) reaches the 90% mark in 3,333 iterations versus 8,000 for Rev-CAGE(0.05), because a larger starting τ_b supplies richer gradient diversity from the first iteration. All three Reverse-CAGE variants reach the same 98.1% ceiling, but they require 34k–41k iterations to reach 98% versus 12k for default CAGE ($\tau_{\max}=3.0$; Table 12): starting high and decreasing wastes no early iterations at low τ . The conclusion is that the issue is not low τ but low τ without escape; a fixed ceiling causes irreversible collapse, whereas an adaptive floor does not. The confidence mechanism is therefore essential in both directions, and starting high (CAGE) is faster than starting low (Reverse-CAGE) while reaching the same accuracy ceiling.

I.2. Sensitivity to the $(\tau_{\max}, \tau_{\min})$ Bounds

We sweep each bound while holding the other fixed (Gumbel-ST+CAGE, $K=64k$, $L=6$, MNIST-Binary, 3 seeds). Tables 12 and 13 report Last-10 accuracy and convergence iterations.

Table 12. $\tau_{\max}$ sweep ($\tau_{\min}=0.5$, $\beta=0.99$ fixed). † marks the paper default.

$\tau_{\max}$	Last-10 (%)	Converge (iters)
1.5	98.15 ± 0.01	6,000 $\pm$ 816
2.0	98.19 $\pm$ 0.05	7,333 $\pm$ 471
3.0 †	97.81 ± 0.07	12,000 $\pm$ 816
4.0	96.69 ± 0.17	17,000 $\pm$ 0
5.0	95.19 ± 0.28	25,000 $\pm$ 0

Table 13. $\tau_{\min}$ sweep ($\tau_{\max}=3.0$, $\beta=0.99$ fixed). † paper default; ‡ $\tau_{\min} \approx \tau_{\max}/2$, where CAGE degenerates to a near-constant τ .

$\tau_{\min}$	Last-10 (%)	Converge (iters)
0.1	97.94 $\pm$ 0.07	11,667 $\pm$ 1,247
0.3	97.82 ± 0.03	11,667 $\pm$ 1,700
0.5 †	97.81 ± 0.07	12,000 $\pm$ 816
0.7	97.61 ± 0.06	12,333 $\pm$ 471
1.0	97.59 ± 0.12	14,000 $\pm$ 1,633
1.5 ‡	97.19 ± 0.21	12,000 $\pm$ 816

Within $\tau_{\max} \in [1.5, 3.0]$ the Last-10 range is 0.38%, and within $\tau_{\min} \in [0.1, 1.0]$ it is 0.35%. CAGE’s sensitivity to its own bounds (0.35–0.38%) is thus over $100\times$ smaller than plain Gumbel-ST’s 47.3% sensitivity to a fixed temperature (Table 2). The paper default $\tau_{\max}=3.0$ sits 0.38% below the peak at $\tau_{\max}=2.0$ and converges about $2\times$ slower; it is a conservative choice that trades a little speed for broader exploration headroom.

J. Cross-Architecture Validation: Mixture-of-Experts

The gap decomposition of Section 3.1 is a structural property of the routing mechanism, not of logic gate networks specifically. To test this directly, we instantiate the same four methods (plus their CAGE variants and Soft-Mix) in a mixture-of-experts (MoE) classifier and measure the selection and computation gaps.

J.1. Setup

We use a 2-layer MoE classifier on MNIST (3 seeds, 20k iterations). Each layer selects one of $K=8$ experts via a learned router (Shazeer et al., 2017; Fedus et al., 2022) with logits $\mathbf{z}^{(l)} = W_r^{(l)} x^{(l)} \in \mathbb{R}^K$, where $W_r^{(l)} \in \mathbb{R}^{K \times d}$ is the router weight matrix for layer $l \in \{1, 2\}$, $x^{(1)} = x$ is the input, and $x^{(2)} = h_1$ is the hidden state produced by layer 1; a linear readout (identical to the one used in our logic gate network classifier) maps h_2 to class logits:

$$x \xrightarrow[\text{router } \mathbf{z}=W_r x]{K=8 \text{ experts}} h_1 \xrightarrow[\text{router } \mathbf{z}=W_r h_1]{K=8 \text{ experts}} h_2 \xrightarrow{\text{readout}} \hat{\mathbf{y}}.$$

We instantiate the experts in three ways: a heterogeneous bank of four MLP and four Transformer blocks ($\text{MLP} \times 4 + \text{Tr} \times 4$), and homogeneous logic-gate experts ($\text{LGN} \times 8$) on both binary and continuous MNIST inputs. The default forward temper-

ature is $\tau=1$; CAGE variants use $(\tau_{\max}, \tau_{\min})=(3.0, 0.5)$.

J.2. Results

Table 14. MoE gap decomposition ($K=8$ experts, MNIST, 3 seeds, 20k iters). A^{hard} is test accuracy under hard argmax selection (Section 3.1). $\text{SelGap} = A^M - A^{\text{soft}}$; $\text{CompGap} = A^{\text{soft}} - A^{\text{hard}}$.

Method	Expert	A^{hard} (%)	SelGap (%)	CompGap (%)
<i>MLP + Transformer experts</i>				
Hard-ST	MLP×4+Tr×4	97.7 ± 0.5	+0.00	+0.00
Gumbel-ST	MLP×4+Tr×4	97.8 ± 0.3	+0.00	+0.00
Hard-ST+CAGE	MLP×4+Tr×4	98.0 ± 0.1	+0.00	+0.00
Gumbel-ST+CAGE	MLP×4+Tr×4	98.1 ± 0.1	+0.00	+0.00
Soft-Mix	MLP×4+Tr×4	88.1 ± 3.9	+10.19	+0.00
<i>LGN experts</i>				
Hard-ST	LGN×8 (binary)	90.1 ± 0.4	+0.00	+0.00
Hard-ST	LGN×8 (cont.)	89.5 ± 0.2	+0.00	+1.37

Table 14 confirms every structural prediction of the framework. Selection gap is exactly zero for all hard-routing methods (Proposition 3) and positive for Soft-Mix (Proposition 4). Computation gap is zero for all routing methods on continuous experts (MLP, Transformer), because these experts have no soft-versus-hard distinction; it becomes positive (+1.37%) only for Hard-ST on LGN experts with continuous input, exactly as expected: computation gap is determined by the selected expert’s computation, not by the routing method (Section 3.1). The same patterns hold for fully homogeneous banks (MLP×8, Transformer×8, $K=8$ identical experts): selection gap is zero for hard routing and positive for Soft-Mix across all configurations. CAGE further improves accuracy in this setting (Gumbel-ST+CAGE 98.1% vs. Gumbel-ST 97.8%, +0.3%) through its confidence-adaptive gradient schedule.

Remark on neural architecture search. The MoE router is a *shared, input-dependent* module ($\mathbf{z} = W_r x$). Neural architecture search instead uses *standalone per-edge* architecture weights α that are directly analogous to the LGN gate logits rather than to the MoE router. Since the same gradient mechanism (Proposition 8) governs both, CAGE is in principle applicable to NAS; we leave a full NAS study to future work (the NAS gap is discussed in Section 2).

K. Input Distribution Analysis

This appendix analyzes how input distribution affects the computation gap between soft and hard logic gates, explaining why computation gap is small in our experiments.

K.1. Gate Definitions

We use product t-norm and probabilistic t-conorm relaxations (Klement et al., 2000; Van Krieken et al., 2022; Petersen et al., 2022) for soft gates, with standard negation ($\neg a \rightarrow 1 - a$). Hard gates use Boolean operations with threshold $\theta = 0.5$. Table 15 lists all 16 gates.

K.2. Computation Gap Bounds

The computation gap for a gate g is the expected absolute difference between soft and hard outputs:

$$\Delta_g = \mathbb{E}_{a,b \sim \mathcal{D}} [|g^{\text{soft}}(a, b) - g^{\text{hard}}(a, b)|]. \quad (44)$$

The gap depends critically on input distribution. Binary inputs $\{0, 1\}$ achieve zero gap since soft and hard gates produce identical outputs. Uniform $[0, 1]$ inputs yield moderate gap ($\sim 20\%$). The upper bound occurs at the decision boundary ($a \approx 0.5$). Table 16 summarizes these bounds.

For Uniform $[0, 1]$ inputs, the gaps can be computed analytically. Unary gates have $\Delta = 0.25$. AND and OR gates have $\Delta = 0.21875$ (by quadrant integration; contributions swap between gates but totals match). XOR gates have $\Delta = 0.167$. Negation preserves gap: $\Delta_{\text{NAND}} = \Delta_{\text{AND}}$, etc. Table 17 groups all 16 gates by gap value.

Table 15. All 16 two-input Boolean gates with soft (differentiable) and hard (Boolean) formulas.

Index	Gate	Boolean	Soft Formula	Hard Formula
0	FALSE	0	0	0
1	TRUE	1	1	1
2	A	a	a	$\mathbb{1}[a > 0.5]$
3	B	b	b	$\mathbb{1}[b > 0.5]$
4	$\neg A$	$\neg a$	$1 - a$	$\mathbb{1}[a \leq 0.5]$
5	$\neg B$	$\neg b$	$1 - b$	$\mathbb{1}[b \leq 0.5]$
6	AND	$a \wedge b$	ab	$\mathbb{1}[a > 0.5] \wedge \mathbb{1}[b > 0.5]$
7	OR	$a \vee b$	$a + b - ab$	$\mathbb{1}[a > 0.5] \vee \mathbb{1}[b > 0.5]$
8	XOR	$a \oplus b$	$a + b - 2ab$	$\mathbb{1}[a > 0.5] \oplus \mathbb{1}[b > 0.5]$
9	NAND	$\neg(a \wedge b)$	$1 - ab$	$\neg(\mathbb{1}[a > 0.5] \wedge \mathbb{1}[b > 0.5])$
10	NOR	$\neg(a \vee b)$	$1 - a - b + ab$	$\neg(\mathbb{1}[a > 0.5] \vee \mathbb{1}[b > 0.5])$
11	XNOR	$\neg(a \oplus b)$	$1 - a - b + 2ab$	$\neg(\mathbb{1}[a > 0.5] \oplus \mathbb{1}[b > 0.5])$
12	$A \wedge \neg B$	$a \wedge \neg b$	$a(1 - b)$	$\mathbb{1}[a > 0.5] \wedge \mathbb{1}[b \leq 0.5]$
13	$\neg A \wedge B$	$\neg a \wedge b$	$(1 - a)b$	$\mathbb{1}[a \leq 0.5] \wedge \mathbb{1}[b > 0.5]$
14	$A \vee \neg B$	$a \vee \neg b$	$1 - b + ab$	$\mathbb{1}[a > 0.5] \vee \mathbb{1}[b \leq 0.5]$
15	$\neg A \vee B$	$\neg a \vee b$	$1 - a + ab$	$\mathbb{1}[a \leq 0.5] \vee \mathbb{1}[b > 0.5]$

Table 16. Computation gap for representative gates under different input distributions.

Distribution	Unary	AND/OR	XOR
Binary $\{0, 1\}$ (lower bound)	0%	0%	0%
MNIST (91% binary-like)	$\sim 2\%$	$\sim 3\%$	$\sim 5\%$
Uniform $[0, 1]$	25%	21.9%	16.7%
At boundary $a=0.5$ (upper)	50%	75%	50%

K.3. MNIST Pixel Distribution

MNIST is highly sparse (Table 18): 80.9% of pixels are exactly zero (background), and only 8.7% fall in the intermediate range $[0.1, 0.9]$. Overall, 91.3% of pixels are binary-like (< 0.1 or > 0.9). This explains why computation gap is small in our experiments.

K.4. Implications

With 91.3% binary-like pixels, MNIST is close to the best-case scenario where binary inputs yield 0% gap. MNIST-Binary achieves exactly zero computation gap, isolating pure selection gap. This explains why our experiments show selection gap as the dominant component: MNIST’s distribution minimizes computation gap, making selection gap the primary differentiator between methods.

For applications with truly continuous inputs (e.g., Uniform $[0, 1]$), computation gap becomes significant ($\sim 20\%$), and inputs concentrated near the decision boundary can reach 50–75% gap.

Table 17. Expected computation gap for all 16 gates on Uniform [0,1] inputs.

Category	Gates	Gap (%)
Constants	FALSE, TRUE	0.0
XOR-type	XOR, XNOR	16.7
AND/OR-type	AND, OR, NAND, NOR, $A \wedge \neg B$, $\neg A \wedge B$, $A \vee \neg B$, $\neg A \vee B$	21.9
Unary	A, B, $\neg A$, $\neg B$	25.0

Table 18. MNIST pixel value distribution.

Pixel Range	MNIST [0,1]	MNIST-Binary
Exactly 0	80.9%	86.7%
(0, 0.1)	1.7%	0.0%
[0.1, 0.9]	8.7%	0.0%
(0.9, 1.0]	8.8%	13.3%
Binary-like (< 0.1 or > 0.9)	91.3%	100.0%